

Highlights

Beyond engine latency: frequency governors and host pipelines in object detection on edge accelerators

Dinh Thuan Nguyen[✉], Lam Phuong Nguyen[✉], Vinh Huy Nguyen[✉], Sy Vu Quang, Mohan Rajesh Elara[✉], Anh Vu Le[✉]

- On Jetson, the stock GPU governor triples serial inference time.
- Host restructuring beats a maximum-clock serial pipeline by 2.4×.
- Sleeping waits down-clock the host and cost 20% of sequential throughput.
- At camera rates, idle power dominates energy while scheduling sets latency.
- Pipeline gains transfer across neural accelerators, but mechanisms differ.

Beyond engine latency: frequency governors and host pipelines in object detection on edge accelerators

Dinh Thuan Nguyen^a, Lam Phuong Nguyen^a, Vinh Huy Nguyen^b, Sy Vu Quang^c, Mohan Rajesh Elara^d and Anh Vu Le^{c,*}

^aFaculty of Electrical and Electronics Engineering, Ton Duc Thang University, Ho Chi Minh City, Vietnam

^bCenter of Visualization and Simulation, Duy Tan University, Da Nang, Vietnam

^cAdvanced Intelligent Technology Research Group, Faculty of Electrical and Electronics Engineering, Ton Duc Thang University, 19 Nguyen Huu Tho Street, Tan Phong Ward, District 7, Ho Chi Minh City 700000, Vietnam

^dROAR Lab, Engineering Product Development, Singapore University of Technology and Design, Singapore

ARTICLE INFO

Keywords:

Edge AI
Object detection
DVFS
Embedded GPU
Energy efficiency
Jetson
Benchmarking methodology

ABSTRACT

The deployment cost of NMS-free detectors is usually equated with engine latency. Here, the full decode-to-detections pipeline of YOLO26n is measured on a Jetson Orin Nano Super, with module input power, clocks, GPU load and COCO accuracy on the 5,000 val2017 images. The FP16 TensorRT engine that takes 4.2 ms back-to-back takes 13.2 ms inside a sequential application, because the stock GPU governor holds the GPU at its minimum clock. A duty-cycle model describes the measurements. Engine time scales with the inverse of the clock (3.7% mean absolute error, fitted on a fixed-frequency sweep), interior-clock operating points lie in an observed GPU-load range of 0.53–0.74, and a sequential pipeline released from the top clock falls to the floor within seconds. Restructuring only the host pipeline, under default governors and with identical predictions, gives 2.4× the throughput of the best sequential pipeline at locked maximum clocks. A sleeping CUDA wait also lets `schedutil` lower the host clock and, through slower submission, the GPU clock. A per-task utilization clamp recovers most of this loss without busy-spinning. On the full pipeline, sleeping and spinning waits perform alike under default and locked clocks. Below capacity, idle power sets the energy per frame and the schedule sets the latency. Replications on a Raspberry Pi 5 with a Hailo-8 NPU and on a Qualcomm QCM6490 show that the effect transfers, while its mechanism moves to the host. Edge detection efficiency should be reported for the whole pipeline under deployment governors.

1. Introduction

Real-time object detection on embedded GPUs is usually evaluated with two numbers per model: COCO accuracy [1] and the latency of the compiled engine in an isolated benchmark. Efficiency research therefore focuses on the model: structured pruning [2, 3], quantization-aware training [4] and distillation [5] are judged by the engine latency they remove per unit of accuracy lost. NMS-free detectors make this framing seem even more natural. YOLOv10 [6] and YOLO26 [7, 8] emit a fixed set of final detections from a one-to-one head, so the host only decodes the image and converts a few hundred output rows. With post-processing nearly gone, the engine benchmark should predict the application.

Measurements on an NVIDIA Jetson Orin Nano Super show that it does not. Run back-to-back, the FP16 TensorRT engine of YOLO26n takes 4.2 ms per inference. In a conventional sequential application that decodes a JPEG, letterboxes it, runs the engine and builds detection rows, the same engine takes 13.2 ms. The model, input and builder are unchanged; the GPU’s duty cycle is not. While the CPU decodes, the GPU idles, and the stock frequency governor holds the GPU at its minimum clock of 306 MHz, 30% of

its maximum. This also affects how compression should be judged. A model 20% cheaper in the benchmark would gain little in such an application: at a fixed clock the saving is diluted by the host stages, and the clock itself does not move.

The study did not start with governors. It first tried to make YOLO26n cheaper through object-aware structured pruning, quantization-aware training (QAT), distillation, compiler-guided pruning of the most expensive head layers, and a per-image adaptive-resolution router. These variants either lost several AP points or did not run faster once compiled, and the router failed verification on held-out images. Only the host pipeline was then restructured. Under the same default governors and with byte-identical predictions, this gave 2.4× the throughput of the best sequential pipeline, even with all of that pipeline’s clocks locked at maximum. The gain over the standard Ultralytics predictor was 6.9×.

This paper is a systematic measurement study of that gap, guided by five research questions:

RQ1 How does the GPU governor’s response to an application’s duty cycle determine the in-application cost of a fixed engine, and can a simple model describe it?

RQ2 Which host-pipeline decisions matter for throughput and energy, how large is each effect, and which of them interact with the governor?

Corresponding author:

E-mail addresses: nguyendinhthuan.st@tdtu.edu.vn (D. T. Nguyen),
nguyenphuonglam.st@tdtu.edu.vn (L. P. Nguyen),
nguyenvinhhuu@tdtu.edu.vn (V. H. Nguyen), vuquangsy@tdtu.edu.vn
(S. V. Quang), rajeshelara@sutd.edu.sg (M. R. Elara),
leanhvu@tdtu.edu.vn (A. V. Le).

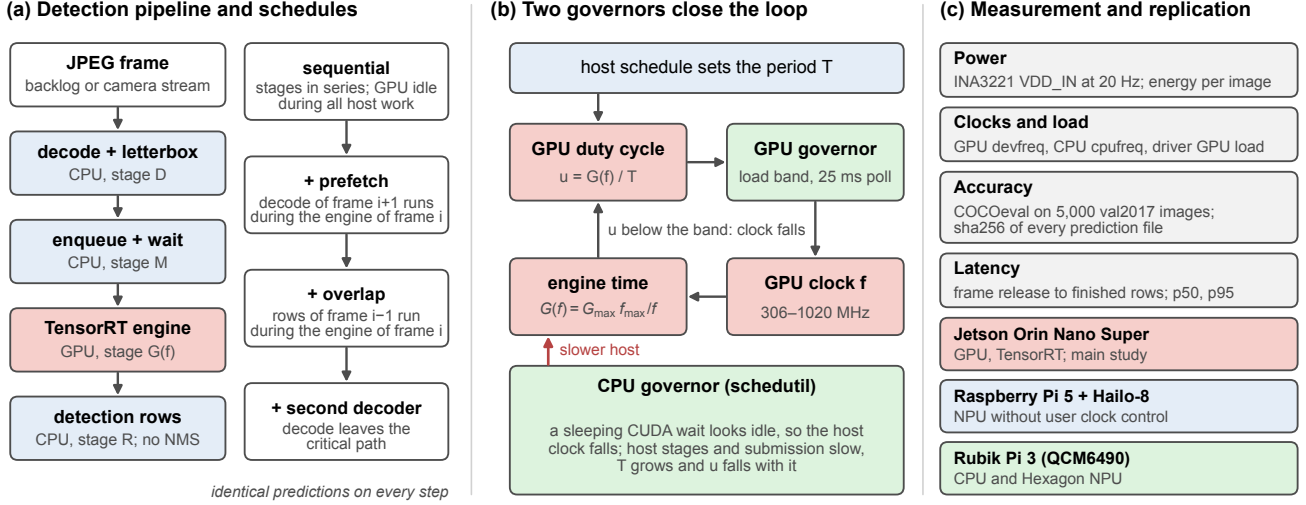


Figure 1: Overview of the study. (a) The detection pipeline, with the stage symbols of Section 3, and the cumulative schedule ladder of Section 6; every scheduling step keeps the predictions byte-identical. (b) The feedback loop described by the duty-cycle model. The host schedule sets the period T and thus the GPU duty cycle u . When u lies below the governor’s load band, the GPU clock falls and engine time grows. A sleeping CUDA wait also lets the CPU governor lower the host clock, which lengthens T . (c) Quantities measured on every run and the three boards of the study (Sections 4 and 9).

RQ3 How do these decisions trade latency against energy when frames arrive at camera rates rather than as a backlog?

RQ4 How do model-level decisions (input resolution, compression, model size) rank once the pipeline is fixed?

RQ5 Do the findings carry over to other NMS-free detectors and to other accelerator architectures?

The contributions are:

1. A duty-cycle model of in-application engine time and energy (Section 3), checked against direct measurements (Section 5). Engine time scales with the inverse of the measured clock. The driver-reported GPU load is consistent with a load-band governor whose observed band is schedule-specific. A sequential pipeline sits at or below the top of that band even at the minimum clock, which is consistent with its never leaving the floor. Splitting energy into an idle floor and a marginal term gives an empirical upper envelope for the measured below-capacity streaming cells (Section 7).
2. A controlled, cumulative pipeline ablation on all 5,000 COCO val2017 images (Section 6). Scheduling variants produce byte-identical predictions, so accuracy is held exactly constant. Each variant is reported with official COCO AP, throughput, input energy, GPU clock and CPU time under both default and locked clocks, and the number of decode workers is swept. On the sequential and prefetch schedules, the choice between two CUDA synchronization calls costs 17–20% of throughput under the default governors. This loss is governor-mediated and drops below 1% with

locked clocks. On the full two-worker schedule the wait policy itself is free: a sleeping event wait matches a spinning one within 1% under default and locked clocks. The 24% that stream synchronization still costs there is a completion-boundary effect, identical at locked clocks. A controlled test traces the governor-mediated loss to the CPU governor: a sleeping wait lets schedutil lower the clock of the host’s frequency domain. Keeping that domain busy recovers 97% of the loss. A per-task utilization clamp [9], set without root privileges, recovers most of it at lower energy. Garbage-collection pauses in a typical evaluation loop also cost a fast pipeline 7.1% of its measured throughput.

3. A streaming evaluation at 15, 30 and 60 fps (Section 7). Energy per frame is set mainly by idle power. The highest-throughput configuration adds one frame period of latency at low rates. An arrival-aware variant removes this cost and keeps p95 latency at or below 41 ms at all three rates.
4. An evaluation of model-level decisions under a fixed, optimized pipeline (Section 8). Lower input resolution buys 27% throughput in a naive pipeline but only 2–4% in an optimized one (6–10% with Python’s cyclic garbage collector disabled). The compression and routing attempts are reported as negative results. On YOLO26s and YOLOv10n, the pipeline effects are comparable to YOLO26n’s (5.6–6.9× over the standard predictor).
5. A replication of the core experiments on two further boards (Section 9): a Raspberry Pi 5 with a Hailo-8 NPU, whose accelerator has no user-visible frequency control, and a Rubik Pi 3 (Qualcomm QCM6490),

measured on both its CPU and its Hexagon NPU. The pipeline ladder transfers ($3.3\times$ on the Pi 5 with byte-identical predictions, $2.7\times$ before the wait-policy change). The Jetson-specific accelerator-governor signature is not directly observed on the NPU stacks, where the identifiable mechanisms shift to the host side. With CPU inference, the ladder flattens and sustained load becomes thermally limited. On the NPU stack, the CPU governor becomes the largest term (57% on the matched overlap rungs) because the scheduler down-clocks the core that feeds the accelerator. That core falls from 2.7 GHz (pinned) to 0.95–1.8 GHz, depending on how little host work the schedule leaves on the critical path.

Fig. 1 gives an overview of the pipeline, the feedback loop and the measurement setup. This paper does not propose a new detector, compression method or governor, and pipelining itself is standard practice. Its contribution is the measurement and its explanation. On this class of device, the interaction between the host pipeline and the frequency governor determines more of the end-to-end cost than the model-level choices that usually receive attention. All scripts, raw power traces and prediction hashes are provided for checking and reproduction (see the data and code availability statement).

2. Related Work

NMS-free detectors. Set-prediction detectors remove non-maximum suppression (NMS) by one-to-one matching [10]; RT-DETR made this regime real-time on GPUs [11]. YOLOv10 introduced consistent dual assignments so that a convolutional YOLO can be deployed with a one-to-one head and no NMS [6], and YOLO26 continues this design with an end-to-end head [7, 8]. With NMS gone, only decode, letterbox and a short top- k row conversion remain on the host. This makes the rest of the pipeline more visible and is why these models are studied here.

Compression for edge detection. Structured channel pruning [2, 3], quantization-aware training [4] and knowledge distillation [5] are the standard tools for making detectors cheaper. They are usually judged by parameter count, FLOPs or isolated engine latency. Here they are run on the same deployment stack and reported in end-to-end units. The negative results do not show that compression is ineffective in general, only that, for a nano-scale NMS-free model on this device, the model is not where the time goes.

Latency prediction, DVFS and thermal control. nn-Meter showed that kernel-level effects make FLOPs a poor latency proxy on edge devices [12]. Tang et al. found that the effect of GPU core and memory frequencies on the time and energy of deep learning workloads differs across networks [13]. Nabavinejad et al. chose batch size and GPU frequency jointly for inference under latency limits [14]. Other DVFS work on mobile and edge processors learns frequency policies that avoid throttling [15], models GPU

latency under DVFS [16], or controls frequency per operator on Jetson-class devices [17]. SparseDVFS is motivated by the antagonistic effect of the stock CPU and GPU governors on a Jetson Orin Nano: the CPU governor down-clocks the host during GPU-heavy phases, so the host is slow exactly when the GPU next requests data [17]. Section 6.3 measures the same coupling inside a detection pipeline and traces it to the CUDA wait policy. Work on 3D mobile games showed that joint CPU and GPU frequency management saves power over independent governors, because the two processors depend on each other’s progress [18]. Closest to the present setting, Karzhaubayeva et al. characterize CNN workloads on a Jetson TX2 and design integrated CPU–GPU DVFS governors for them [19]. Lotus manages CPU and GPU frequencies jointly for object detectors on a Jetson Orin Nano to stabilize latency under thermal variation [20]. Kang shows that latency estimators for deadline-aware governors on Jetson Orin must condition on the memory clock [21]. All of these works design or evaluate frequency-control policies that set frequencies, or batch sizes, from outside the application. This work instead keeps the stock governors and asks how an application’s duty cycle interacts with them. Utilization clamping [9], a standard kernel mechanism, is tested as a remedy for the CPU–GPU coupling; it needed no root privileges on the test system.

Characterization of edge inference. Mittal surveyed optimized deep learning on Jetson devices, covering precision, framework and power-mode choices [22]. Hadidi et al. compared frameworks and devices for edge inference and found that, for the same model, the framework changes inference time and energy considerably [23]. These studies measure the model or its framework. This work holds the engine fixed and changes only host code and governor settings.

End-to-end costs outside the model. The “AI tax” study showed that pre- and post-processing can dominate accelerated AI applications [24]. On Jetson, JPEG decode and pre-processing have been offloaded to dedicated units [25], and training pipelines have been characterized for I/O pipelining and power modes [26]. The latency distribution of a YOLO pipeline on Jetson Orin NX has also been modeled stage by stage as a function of image resolution [27]. MLPerf Inference standardizes measurement scenarios, separating offline throughput from server and single-stream latency [28]. For autonomous-driving detection, R-TOD analyzes and minimizes the end-to-end delay from frame exposure to reported detections. It restructures capture and pipelining (on-demand capture, zero-slack and contention-free pipelines) without changing the network [29]. The arrival-aware variant used here (Section 7) belongs to the same family; this work measures its interaction with the stock governors and the idle floor rather than worst-case delay. What this work adds is a measured account, on one device and stack, of how the governor’s response to the application duty cycle changes the apparent cost of the *model*, and of how this reorders deployment decisions such as compression and resolution.

Adaptive resolution. Choosing input scale per image or frame can save compute in detection [30, 31]. The frozen

Table 1

Positioning against the closest measurement studies. “Freq.” is how frequency enters the experiment: set directly by the experimenter, replaced by a newly designed governor (“new gov.”), or left to the stock governor and observed (“modif.”: a modified stock governor; “ucl.”: per-task utilization clamping). “Pipe.”: the whole decode-to-detections pipeline is measured, not only the model. “Acc.”: accuracy is held constant or verified per variant. “Stream.”: camera-rate arrivals are measured, not only backlog throughput. SLO: service-level objective. A dash means the work does not address the aspect.

Work	Workload	Freq.	Pipe.	Acc.	Stream.
Tang et al. [13]	DNN time/energy	direct	—	—	—
Nabavinejad et al. [14]	batching w/ SLOs	direct	—	—	—
SparseDVFS [17]	per-operator DVFS	direct	—	yes	—
Pathania et al. [18]	3D mobile games	modif.	partial	—	—
Karzhaubayeva [19]	CNNs on TX2	new gov.	—	—	—
Lotus [20]	detec., Orin Nano	new gov.	—	—	—
Kang [21]	gov. latency est.	direct	—	—	—
AI tax [24]	pre/post-processing	—	yes	—	—
R-TOD [29]	end-to-end delay	—	yes	yes	yes
Stage-wise latency [27]	YOLO on Orin NX	stock	yes	—	—
This work	detection, 3 boards	stock+ucl.	yes	yes	yes

pre-inference router used here belongs to this family. It is reported because it failed held-out verification and because its expected saving is smaller than the measured pipeline effects.

Table 1 summarizes the positioning. No novelty is claimed for pipelining, preprocessing or DVFS per se. The gap addressed is the interaction with the *stock* governors under the application’s own schedule and arrival process, with accuracy held exactly constant so that the timing effects are attributable.

3. A duty-cycle model of the detection pipeline

A deliberately simple model organizes the measurements. Its three parts describe how engine time depends on the GPU clock, how each pipeline schedule sets the GPU’s duty cycle, and how the governor picks a clock for that duty cycle. A fourth relation splits energy into an idle floor and a marginal part. Fig. 1(b) summarizes the loop. Each part is checked against measurements in Sections 5 to 7.

Pipeline stages. For each image, the CPU runs a decode-and-letterbox stage D and, for float-input engines only, a host normalization N . The GPU runs the engine G , including copies, which take under 1 ms on this unified-memory device. A row construction stage R converts the 300 output rows to image coordinates. Let M denote the remaining per-image work of the main thread (buffer fill, enqueue, synchronization).

Engine time. For a fixed engine, the GPU time at clock f is modeled as

$$G(f) = G_{\max} \frac{f_{\max}}{f}, \quad (1)$$

where $G_{\max}$ is the time at the maximum clock $f_{\max}$. This assumes the engine is limited by the GPU clock and not

by memory. For variable-clock runs, the time-weighted harmonic mean of the sampled clock is used as an approximate summary consistent with Eq. (1).

Schedules and duty cycle. The period T between completed images, and thus throughput $X = 1/T$, depends on which stages lie on the critical path. For k decode workers:

$$T_{\text{seq}} = D + N + G(f) + R + M, \quad (2)$$

$$T_{\text{prefetch}} \approx \max\left(\frac{D+N}{k}, G(f) + R + M\right), \quad (3)$$

$$T_{\text{overlap}} \approx \max\left(\frac{D+N}{k}, R + M, G(f)\right). \quad (4)$$

The GPU busy fraction is $u = G(f)/T$. In the sequential schedule every host millisecond is GPU idle time. Eq. (3) describes the prefetch transition: host stages leave the critical path, and the period becomes the maximum of the worker rate and the GPU-serial part. The stage times in it are clock-dependent, so it serves as a structure rather than a fitted predictor. Section 5 measures them at a down-clocked host; at the higher clocks of the prefetch rung, the decode rate is correspondingly higher. Overlap also removes R (Eq. (4)).

Governor. The GPU governor (nvhost_podgov) ships in NVIDIA’s public Jetson Linux kernel sources [32]. Its `devfreq sysfs` node on the test device exposes only a 25 ms polling interval and no load thresholds. The thresholds were not derived from the source; the governor is instead characterized empirically as a load-band controller. The driver exposes the load it acts on through `sysfs`, so the busy fraction u can be checked directly (Section 5). The governor raises f while u exceeds an upper threshold u_{hi} , lowers it while u is below a lower threshold u_{lo} , and is clamped to $[f_{\min}, f_{\max}]$. A schedule therefore settles either at an interior clock with $u \in [u_{\text{lo}}, u_{\text{hi}}]$ or at one of the clamps. The measurements show a fixed point that this view implies. In Eq. (2), u increases as f decreases, because a slower GPU occupies a larger share of the period. If, even at $f_{\min}$, the sequential pipeline’s busy fraction $G(f_{\min})/T_{\text{seq}}(f_{\min})$ stays below u_{hi} , the governor has no reason to raise the clock, and the application runs at the minimum clock indefinitely, even though the GPU is busy about half of the time (measured loads 0.46–0.66 at the floor, Section 5).

Energy. Let P_{idle} be the module input power with no workload under a given clock policy, and P_{board} the mean power during a run. Energy per image splits as

$$E = \frac{P_{\text{board}}}{X} = \underbrace{\frac{P_{\text{idle}}}{X}}_{\text{idle reference}} + \underbrace{\frac{P_{\text{board}} - P_{\text{idle}}}{X}}_{E_{\text{marg}}}. \quad (5)$$

The first term is an accounting reference: the idle draw of the board in the measured power state, amortized over throughput. It is not a universal lower bound, since a different sleep or power-gating regime could go below it. On a backlog, a faster schedule shrinks the idle-reference term. For a camera that delivers frames at a fixed rate r below the pipeline’s capacity, the throughput is $X = r$ regardless of schedule, so

$$E(r) \approx \frac{P_{\text{idle}}}{r} + E_{\text{marg}}. \quad (6)$$

Table 2

Platform and software. BSP is the board support package.

Board	Jetson Orin Nano Super Developer Kit, 8 GB module
CPU	6× Cortex-A78AE, two frequency domains (CPUs 0–3, 4–5), 115–1728 MHz
GPU	Ampere, 1024 cores, 306–1020 MHz
Power mode	MAXN_SUPER
OS / BSP	Ubuntu 22.04, Jetson Linux R36.5.2
Kernel	5.15.199-tegra
Cooling	fan under thermal control (nvfancontrol); locked runs pin CPU, GPU and EMC clocks, not the fan
CUDA / TensorRT	12.6 / 10.3.0
Detector software	Ultralytics 8.4.172
Baseline runtime	PyTorch 2.5.0 (NVIDIA JetPack build)
Host libraries	Python 3.10.12, OpenCV 4.10.0, NumPy 1.21.5 (custom), 1.26.4 (Ultralytics)
Power sensor	INA3221 VDD_IN (module input), 20 Hz

The schedule then affects energy only through E_{marg} , so pipeline optimization should yield large energy savings on a backlog but only small ones at fixed camera rates. Section 7 tests this prediction and finds that Eq. (6) with the backlog E_{marg} substituted is an empirical upper envelope for the below-capacity cells, not an exact prediction.

4. Experimental methodology

4.1. Platform and software

Table 2 lists the platform. “Default” means the stock governors: `nvhos_tpodgov` for the GPU (eight steps from 306 to 1020 MHz) and `schedutil` for the CPU. “Locked” means `jetson_clocks`, which pins the CPU, GPU and memory controller (EMC) clocks to their maxima [33]. Readback at ≈ 4 Hz, idle and under a 15 s inference load, confirmed the EMC pin (constant 3199 MHz). Each locked phase saves the default configuration to a fresh file and restores it afterwards; governor and clock limits were read back after every phase. The two NumPy versions in Table 2 differ because Ultralytics 8.4.172 requires the newer release. This affects only the short COCO row-conversion step, whose code is identical in the custom pipelines and the baseline.

4.2. Replication platforms

Two further boards (Section 9) rerun the core protocol with the smallest changes their stacks require. On the Raspberry Pi 5, the model is the Hailo Model Zoo’s precompiled HEF (Hailo Executable Format) `yo1ov8n_h8.hef` [34] for the Hailo-8 (INT8, NMS on chip, score threshold 0.2 fixed at compile time), run through HailoRT 4.23.0 asynchronous pipelines with the same prefetch/overlap/spin switches as the TensorRT pipeline. The replication tables use short rung names: `seq` is the sequential pipeline, `prefetch` adds decode prefetch, `full w1/w2/w3` add post-processing overlap with one, two or three decode workers, and `spin` replaces the sleeping wait. Pi 5 power is the rail sum of its power-management IC (PMIC), which covers the system-on-chip (SoC) but not the

5 V loads (the Hailo-8 itself, the fan, USB), so Pi energies are SoC-side lower bounds.

The Rubik Pi 3 (Ubuntu 24.04.4, kernel 6.8) is measured with two stacks. The CPU stack is ONNX Runtime 1.30.0 (default thread settings) [35] on the same YOLO26n ONNX graph as the Jetson float engine. The NPU stack runs an INT8 QNN context binary of the same graph on the Hexagon HTP, with the standard (1,84,8400) head and host NMS at the val settings. The board’s QNN 2.46 predates the QAI RT 2.49.40-built context, so the context runs through the bundled runtime of the `qai-appbuilder 2.50.40` wheel. This userspace is not robust under concurrent calls, so all NPU inference is serialized on one thread and host stages overlap around it. The board has no trustworthy power sensor; its sampler logs only the three CPU frequency domains and the SoC thermal zone, at an effective 13–18 Hz rather than 20 Hz. The Pi sampler reads the PMIC’s SoC-side rails at an effective 10 Hz. Energy integrals are trapezoids over the actual sample timestamps. The HTP stack is bit-deterministic (hash-verified, as on the Jetson and Pi). The multi-threaded CPU stack emits rows in nondeterministic order, so its equivalence is checked on sorted prediction rows and by COCOeval on each rung’s saved predictions.

4.3. Models and engines

The main model is the pretrained YOLO26n checkpoint [7] (2.57 M parameters); generality is tested with YOLO26s (10.0 M) and YOLOv10n (2.78 M) [6]. All are exported to ONNX with `nms=False`, which yields a (1,300,6) tensor of boxes, scores and class indices with no NMS. Ultralytics 8.4.172 loads the YOLO26 head with the one-to-many branch by default, so `nms=False` was verified to select the one-to-one (end-to-end) branch in both export and prediction. Every engine is a static batch-1 FP16 TensorRT engine built on the device with builder optimization level 3 [36], except the compression-screening engines of Section 8, built at level 0 (level 3 reference rows are marked in Table A.1). Two engines are built from each graph:

- a *float* engine that takes a normalized $1 \times 3 \times S \times S$ FP32 RGB tensor, and
- a *uint8* engine whose graph begins with four ONNX operators (Cast, a BGR→RGB Gather, Transpose, division by 255) and takes the letterboxed $S \times S \times 3$ uint8 BGR image directly; its ONNX Runtime outputs are identical to the float graph’s (maximum absolute difference 0 on test images).

The Ultralytics baseline uses an engine exported on the device by Ultralytics from the same checkpoint (`half=True`, `nms=False`) with the TensorRT builder defaults (optimization level 3, as above). It runs through `YOLO.predict` with confidence threshold 0.001 and at most 300 detections, as the custom pipelines do.

4.4. Pipelines

All custom pipelines share one implementation and differ only in the switches below, which define the cumulative ladder of Section 6. Each goes from a JPEG file to COCO-format rows: OpenCV decode, letterbox to $S \times S$ with value-114 padding, copy into a pinned host buffer, TensorRT execution, output copy, and vectorized conversion of rows with confidence ≥ 0.001 to original-image coordinates.

- *Input type*: float input with host normalization, or uint8 input with in-graph preprocessing.
- *Synchronization*: `cudaStreamSynchronize` on the inference stream, or `cudaEventSynchronize` on an event recorded after the output copy. Section 6.3 also sets the CUDA device scheduling flag (auto, spin or blocking-sync) before context creation.
- *Prefetch*: one or two worker threads decode and letterbox upcoming images while the current image is processed.
- *Overlap*: double-buffered I/O, so rows of image $i-1$ are built while image i runs on the GPU. The *arrival-aware* variant (Section 7) finishes the in-flight image immediately if the next frame is not yet available.
- *Garbage collection*: Python’s cyclic collector runs by default; one diagnostic variant (Section 6.2) disables it for the timed window.

4.5. Workloads

Backlog: all images are available at once and processed as fast as possible, giving throughput and energy per image. *Streaming*: frames become available at a fixed 15, 30 or 60 fps over the first 1,000 val2017 images. Frame latency (Table 3) includes any queueing when the pipeline cannot keep up. The median (p50) and the p95 are reported; with 1,000-frame streams the p99 estimate rests on ten samples, and p95 is the highest quantile considered stable at this size. In both workloads the first 20 images are a warm-up and are excluded. In streaming runs the arrival schedule is re-anchored at the end of the warm-up, so the timed window starts with an empty queue. JPEG files are read once before each phase, so all decodes come from the page cache.

4.6. Measurement boundaries

Table 3 fixes the boundaries of the recurring time quantities. The model’s G (Section 3) is the device-side kernel cost, measured as *GPU compute* in the engine sweep and as *enqueue-to-done* inside the pipelines. On this unified-memory board the two differ only by the launch and copy path: 0.1–0.4 ms at the top clock (4.25 ms device-side versus 4.33–4.65 ms enqueue-to-done for the same locked engine). At low clocks the launch path itself stretches (Section 6.3). The M term of Section 3 is host work outside G on the pipeline’s critical path and never double-counts waiting already inside an enqueue-to-done span.

Table 3

Measurement boundaries used throughout. The terms differ in the host work and queueing they span and are never equated.

Term	Span (start → end)
GPU compute (<code>trtexec</code>)	device-side execution of one inference; CUDA events, no host work
Enqueue-to-done ("GPU ms" in tables)	H2D enqueue → completion of its D2H; includes GPU queueing
Accelerator call ("infer", NPU tables)	submit → completion of one NPU call; incl. queue wait in overlaps
Frame latency (p50/p95, "latency")	frame availability → finished COCO rows; backlog: decode start
Completion period	$1/\text{throughput}$ over timed window
Energy per image	$\int P dt$ over window ÷ timed images

4.7. Power, clock and thermal measurement

A separate process samples `VDD_IN` voltage and current, the GPU `devfreq` clock, the highest CPU core clock and the junction temperature (`tj-thermal`) at 20 Hz. Energy is the trapezoidal integral of power over each run’s wall-clock timed window. `VDD_IN` is the module’s main input rail (INA3221 monitor); carrier-board loads (fan, USB peripherals, display) sit on other rails and are excluded. Idle power was measured twice per clock policy over 60 s windows (Section 6). Unless stated otherwise, reported GPU clocks are arithmetic means of the samples; Eq. (1) uses the harmonic mean. To check sampler overhead, the sequential and full pipelines were each run twice with and without the power sampler on the first 2,000 val2017 images, with a lighter GPU load and clock logger running in both cases. Mean throughput differed by less than 0.2% (40.2 versus 40.2 and 209.7 versus 209.4 images/s), well within run-to-run variation.

4.8. Accuracy protocol and data hygiene

Accuracy is official `pycocotools` COCOeval (bbox, maxDets 100) on all 5,000 COCO val2017 images [1], reported as AP (IoU 0.50:0.95), AP_{50} (IoU 0.50) where noted, and AP_S (small objects). Every prediction file is hashed. Pipelines that differ only in scheduling produce byte-identical predictions in every repeat and under both clock policies, so their accuracy is exactly constant. Every development decision (compression screening, router fitting, router threshold) used two disjoint 1,024-image subsets of COCO train2017: a development split and an independent, hash-verified verification split. The pretrained checkpoint was itself trained on train2017, so these splits are not unseen by the base model; they are held out only from this paper’s fitting and selection, their absolute AP is optimistic, and they serve only for relative comparisons between variants. val2017 was used only for the measurements in this paper.

4.9. Use of AI assistance in the measurement process

An AI coding agent (Factory Droid) ran the campaign scripts on the boards, transferred the traces and drafted the aggregation and plotting code. All experiments, protocols and analysis decisions were made by the authors, and every

script was reviewed before it ran. All tables and figures are regenerated from the archived raw traces by the published scripts (Appendix B).

4.10. Repetitions and statistics

Each backlog configuration on YOLO26n is repeated three times per clock policy. Resolution, generality, streaming and scheduling-flag experiments are repeated twice, and locked-clock generality runs once. Follow-up experiments from later sessions used the same software and protocol. Driver-reported GPU load was logged in one run per pipeline. The utilization-clamping, GIL switch-interval, minimum-frequency CPU clamp and float-engine zero-gap `trtexec` cells have two runs each, and the fixed-frequency sweep has three 15 s runs per devfreq step (Section 5). These runs also logged GPU load, GPU clock and the clock of CPUs 0–3 at 20 Hz. Configuration order is rotated between repeats, with a 10 s pause between runs, and no other GPU work runs during a measurement. Means and full ranges (minimum to maximum) are reported rather than confidence intervals, because two or three repeats are too few for a meaningful interval. Run-to-run ranges are small (median 0.8% of the mean for backlog throughput). The held-out router test uses a paired image bootstrap.

5. RQ1: The governor sets the in-application engine time

5.1. Stage attribution in a sequential application

Each stage of a conventional sequential application was first timed with CUDA events (float-input YOLO26n engine at 640, 1,004 images of the train2017 verification split). The medians were 7.5 ms for JPEG decode, 3.9 ms for letterbox and normalization, 0.65 ms for the pinned host-to-device copy, 12.4 ms for TensorRT execution and 0.01 ms for the output copy. With row construction and Python-side dispatch (≈ 1.5 ms), the median total is 26.0 ms for the OpenCV-based row path (the fallback converter of the accuracy runs is excluded). The default governors held the CPU domain near 955 MHz, so these are host-stage times of a down-clocked host. A clocked repeat (domain logged at 20 Hz, 837 MHz mean) reproduced every stage median within 7% (decode 8.0 ms, letterbox 4.2 ms, compute 12.9 ms, total 26.9 ms). Copies are negligible on this unified-memory device, and the NMS-free output needs no suppression. By this account the model takes about half of the application’s time and is the obvious optimization target.

The same engine, however, runs in 4.1 ms back-to-back in `trtexec` (zero gap, two 15 s runs, medians 4.10 and 4.13 ms), and the uint8-input engine of Table 4 in 4.2 ms. On val2017, every sequential pipeline of the ladder (Section 6) records a mean GPU clock of exactly 306 MHz, the governor’s floor, and a median enqueue-to-completion time of 13.2–13.3 ms for both input types, 3.1–3.2 $\times$ the respective back-to-back time.

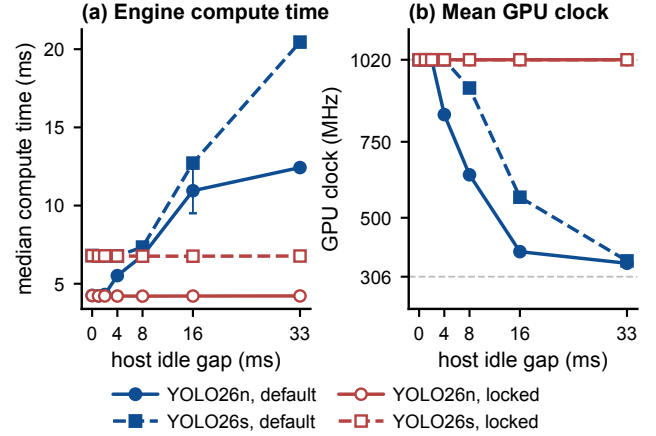


Figure 2: Median GPU compute time of the uint8 FP16 engines in `trtexec` (left) and mean sampled GPU clock (right) versus a forced idle gap between inferences. Mean of two 15 s runs per point. Default governors (blue, filled markers) versus `jetson_clocks` (red, open markers); circles and solid lines show YOLO26n, squares and dashed lines YOLO26s. The gap is host idle time (`-idleTime`), not a frame period.

5.2. An idle-gap sweep isolates the governor

To isolate the governor, the uint8 engines were run in `trtexec` with a forced idle gap between inferences, twice for 15 s per point (Fig. 2, Table 4). With locked clocks, YOLO26n’s median compute time is 4.20–4.27 ms at every gap. With default governors it stays at 4.2–4.3 ms up to a 2 ms gap, then grows to 5.5 ms at 4 ms, 6.8 ms at 8 ms and 12.4 ms at 33 ms. YOLO26s, which keeps the GPU busy longer per inference, holds its 6.8 ms up to a 4 ms gap and rises to 20.5 ms at 33 ms. The gap is host idle time: at 33 ms the achieved rate is about 26.7 inferences/s, with no camera-style arrival process, and because `trtexec` pipelines its enqueues the GPU-idle interval is shorter than the requested sleep (Table 4). Along this axis, a host that leaves ≈ 30 ms between inferences pays about 3 $\times$ the minimum compute time per frame under the default governor. Camera-rate arrival is measured in Section 7.

5.3. Validating the duty-cycle model

Engine time. Table 4 compares the measured compute time with the prediction of Eq. (1), which uses $G_{\max}$ from the locked runs and the harmonic-mean clock sampled during each run. Over the 14 down-clocked sweep runs, the mean absolute error is 4.8%, with signed per-cell errors from -6.7% (YOLO26n, 4 ms gap) to $+11.0\%$ (YOLO26n, 16 ms gap). The largest error (23%) occurs in one of the two 16 ms-gap YOLO26n runs, whose median compute time differs from its repeat’s by 30% although their harmonic-mean clocks differ by only 8%. This suggests clock changes faster than the 20 Hz sampling resolves; independent measurements put the GPU clock-actuation lag at about 5 ms on this SoC family [21].

For the non-overlapped application rungs of Table 6, Eq. (1) overestimates the measured enqueue-to-completion

Table 4

Idle-gap sweep (trtexec, uint8 FP16 engines, mean of two 15 s runs). Default governors: measured median compute time, harmonic-mean GPU clock $\bar{f}$, and the prediction $G_{\max}f_{\max}/\bar{f}$ of Eq. (1). “Lock” is the measured median with jetson_clocks. The gap axis is the requested host idle time (`-idleTime`), not a frame period. Because trtexec pipelines its enqueues, the achieved rate (last row pair) implies a mean GPU-idle interval shorter than the requested sleep. Where the host requests 4, 8, 16 and 33 ms, this interval is 3.0, 5.7, 9.4 and 25.4 ms for YOLO26n and 1.5, 4.9, 8.1 and 18.0 ms for YOLO26s.

Gap (ms)	YOLO26n				YOLO26s			
	ms	$\bar{f}$	pred.	lock	ms	$\bar{f}$	pred.	lock
0	4.25	1020	4.22	4.25	6.80	1020	6.78	6.80
1	4.23	1020	4.22	4.22	6.79	1020	6.78	6.78
2	4.31	1020	4.22	4.22	6.78	1020	6.78	6.78
4	5.52	836	5.15	4.21	6.77	1020	6.78	6.78
8	6.80	632	6.82	4.21	7.35	926	7.46	6.76
16	10.95	355	12.15	4.21	12.72	550	12.57	6.76
33	12.43	323	13.33	4.22	20.45	327	21.12	6.77

achieved inf/s, YOLO26n: 235, 236, 169, 118, 81, 49, 27
 achieved inf/s, YOLO26s: 147, 147, 147, 121, 82, 50, 27

time by 0.4–11.5%. Part of this bias is expected: Eq. (1) attributes all engine time to the GPU clock, with no clock-insensitive component (memory-bound kernels, launch overhead). The window’s harmonic-mean clock also includes host-only intervals, during which the clock is lower. In the load-logging session, re-weighting the clock by the driver’s per-sample GPU load reduces the prefetch rungs’ overestimate from +19–36% to +9–13%, so much of the bias where the clock moves is a sampling artefact of the window average. The sequential rungs, pinned at 306 MHz throughout, still show a 5–7% overestimate, the cleaner estimate of the clock-insensitive share. Eq. (1) is therefore treated as a first-order description with a ~5% residual, not as an exact law.

A fixed-frequency sweep avoids window averaging. With the GPU’s devfreq domain pinned at each of its eight steps (CPU and memory clocks at their shipped settings), the uint8 engine at zero gap takes from 12.49 ms at 306 MHz to 4.25 ms at 1020 MHz (Table 5). A pure inverse fit $G = b/f$, with b fitted by least squares, reproduces these medians with a mean absolute error of 3.7% but a systematic residual, positive at the floor and negative at the top clock. The anchored form of Eq. (1) ($b = G_{\max}f_{\max}$) is less accurate (6.6% mean absolute error, always overestimating below the top clock). The two-term fit $G(f) = a + b/f$ removes the structure ($a = 0.63$ ms, $b = 3.63$ GHz·ms, mean absolute error 0.7%, all residuals within $\pm 1.5\%$). Adding the ≈ 0.7 ms launch path to the pinned 306 MHz time reproduces the sequential pipeline’s 13.2 ms enqueue-to-completion.

The memory (EMC) clock also scales with load on this SoC and is a possible confound; prior work reports memory-clock effects of 11–48% on edge-inference latency [21]. The sweep was therefore rerun with three repeats per step and

Table 5

Fixed-frequency sweep: the uint8 YOLO26n engine at zero idle gap with the GPU pinned at each devfreq step (three 15 s trtexec runs per step; repeat medians agree to 1.1%). The memory clock is logged (constant 2133 MHz at 306–510 MHz, bistable 2133–3199 MHz from 612 MHz up). $G(f) = b/f$ uses the least-squares $b = 3.94$ GHz·ms fitted to the eight points. Anchoring $b = G_{\max}f_{\max}$ at the top point instead, as Eq. (1) does, raises the mean absolute error to 6.6%. The two-term fit adds a clock-insensitive a . Errors are signed, (predicted – measured)/measured.

f (MHz)	measured (ms)	b/f (%)	$a+b/f$ (%)
306	12.49	+3.1	+0.1
408	9.54	+1.2	–0.1
510	7.86	–1.7	–1.3
612	6.48	–0.6	+1.4
714	5.68	–2.8	+0.7
816	5.05	–4.5	+0.6
918	4.59	–6.5	0.0
1020	4.25	–9.1	–1.4

the EMC clock logged at ≈ 4 Hz, plus two control runs with the EMC governor fully unlocked. An attempted EMC lock did not hold; on the same SoC family, EMC clock control is documented to fail silently on these board support packages unless a second lock bit is asserted [21]. The confound is therefore controlled by measurement. The logged memory clock is bistable across the GPU steps (Table 5). Fitting $a+b/f$ to only the EMC-constant points changes b by 2% (3.55 versus 3.63 GHz·ms), and the unlocked control runs reproduce the pinned medians within 0.4%. The inverse trend is thus primarily a GPU-clock effect and cannot be explained by memory-clock scaling alone.

Governor operating points. The busy fraction u of Section 3 can be estimated from timing, but the GPU driver also reports its own load (per mille, through sysfs). In a separate session, the eight pipeline rungs were rerun on the first 2,000 val2017 images with this load and the GPU and CPU clocks logged at 20 Hz. The measured load agrees with the timing estimate within 0.05 for every rung (the estimate is 0.01–0.05 higher) and is used for the application rungs below.

Fig. 3 plots every operating point as GPU load against harmonic-mean clock. Whenever the governor settles clearly inside its range (400–1000 MHz), the load lies between 0.53 and 0.74: 0.53–0.64 for the trtexec sweep (timing estimate) and 0.64–0.74 for the application rungs (measured). Two caveats apply. The two sources use different estimators, so the lower edge cannot be separated from the timing estimate’s high bias. The 0.74 upper edge comes from a rung settled at 993 MHz, close to the 1020 MHz clamp, so the true upper threshold may lie higher. With these caveats, this is the load-band behavior assumed in Section 3. The sequential rungs sit at the floor with measured loads of 0.46–0.66, at or below the observed band’s top, consistent with the governor never raising the clock (the fixed point of

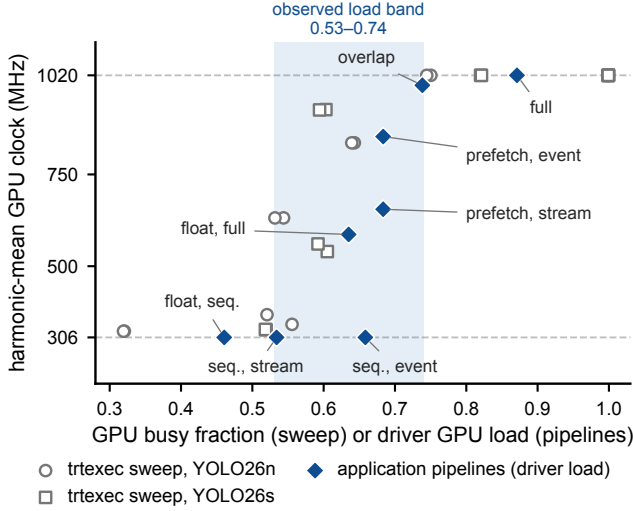


Figure 3: Governor operating points: GPU busy fraction or measured load versus harmonic-mean GPU clock. Open markers: trtexec idle-gap sweep, busy fraction estimated from timing. Diamonds: default-governor application runs (first 2,000 val2017 images), load measured by the GPU driver. The shaded band (0.53–0.74) is the observed operating-load range. It contains every operating point with a clock between 400 and 1000 MHz and is not an identified pair of controller thresholds.

Section 3). Removing host stages from the GPU’s critical path pushes the load to the top of the band or above, and the clock follows. Prefetch with event synchronization settles at 853 MHz (harmonic mean) in the load-logging session and 881–891 MHz across the three backlog repeats; its stream-synchronized variant stays at 655 MHz here and at 631–694 MHz in the blocking-sync runs of Section 6.3. The full schedule, at a measured load of 0.87 (timing estimate 0.90), stays at 1020 MHz.

The floor is the only stable point observed. Whether a load-band controller holds a sequential pipeline at a high clock if it starts there was tested twice: the sequential stream-synchronized pipeline was started with locked clocks and the default governor restored after 10 s. At 1020 MHz the load was only 0.35–0.36, below the band. Within 0.25 s of the release the governor began to step down (1020, 714, 612, 510, 408 MHz), held 408 MHz for about 2.5 s at a load of about 0.5, and reached 306 MHz 3.3–3.4 s after the release, where it stayed at a load of 0.52–0.53. No run showed a second operating point. The model does not exclude interior equilibria (a pipeline whose load stops responding to the clock could settle elsewhere), but for this workload the release lands at the floor from every starting clock tested.

Clock and power traces. In Fig. 4, both sequential variants stay at 306 MHz, prefetch climbs in steps from the floor and then moves between about 700 and 1020 MHz, and the full schedule is pinned at 1020 MHz. Input power follows the clock. Both optimized schedules also show short power dips whose spacing grows over the run: from about 1.9 to 4.3 s for the full schedule, and at longer, less regular intervals (up to about 7 s) for prefetch. The dips were traced

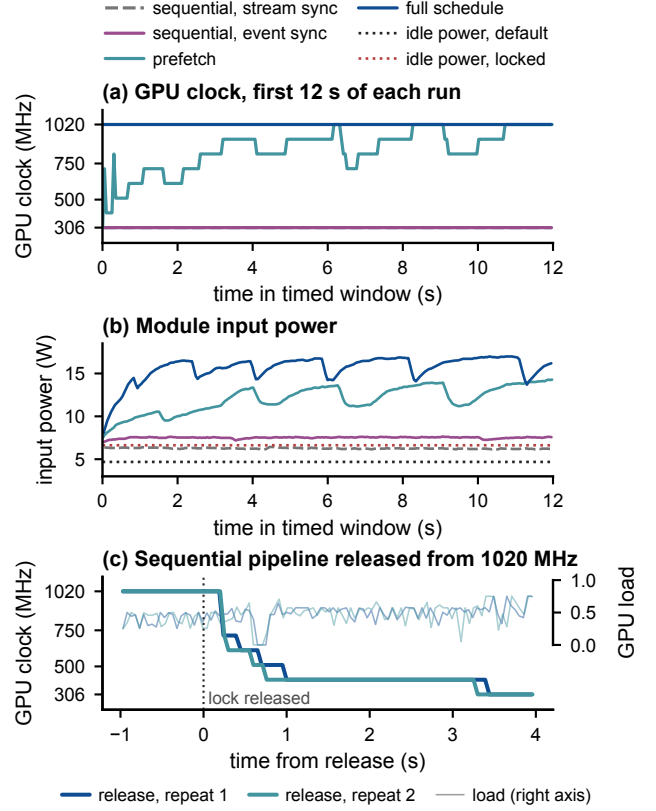


Figure 4: (a) GPU clock and (b) module input power during the first 12 s of four YOLO26n application runs on val2017 under default governors (20 Hz samples); the two sequential traces coincide at 306 MHz in the top panel. (c) The lock-then-release experiment. The sequential pipeline is started with the GPU clock locked at 1020 MHz, and the lock is released at $t=0$ (dotted line). Clock (thick lines) and driver GPU load (thin lines, right axis) of two repeats fall to the 306 MHz floor within about 3.4 s.

to Python’s cyclic garbage collector, whose pauses grow with the list of detection rows; their effect over whole runs is quantified in Section 6.

Answer to RQ1. Under the default governor, the in-application cost of a fixed engine is set by the application’s GPU duty cycle. Engine time follows the inverse of the clock (Eq. (1)). Every operating point observed inside the governor’s range sits in a GPU-load band of 0.53–0.74, consistent with the load-band behavior of Section 3, whose exact thresholds are not identified. The only other observed state is the floor. A sequential pipeline reaches it from any starting clock and stays there, which makes its engine about 3× slower than the benchmark, a larger effect than any engine-time change produced by the compression methods tested (Section 8).

Beyond engine latency

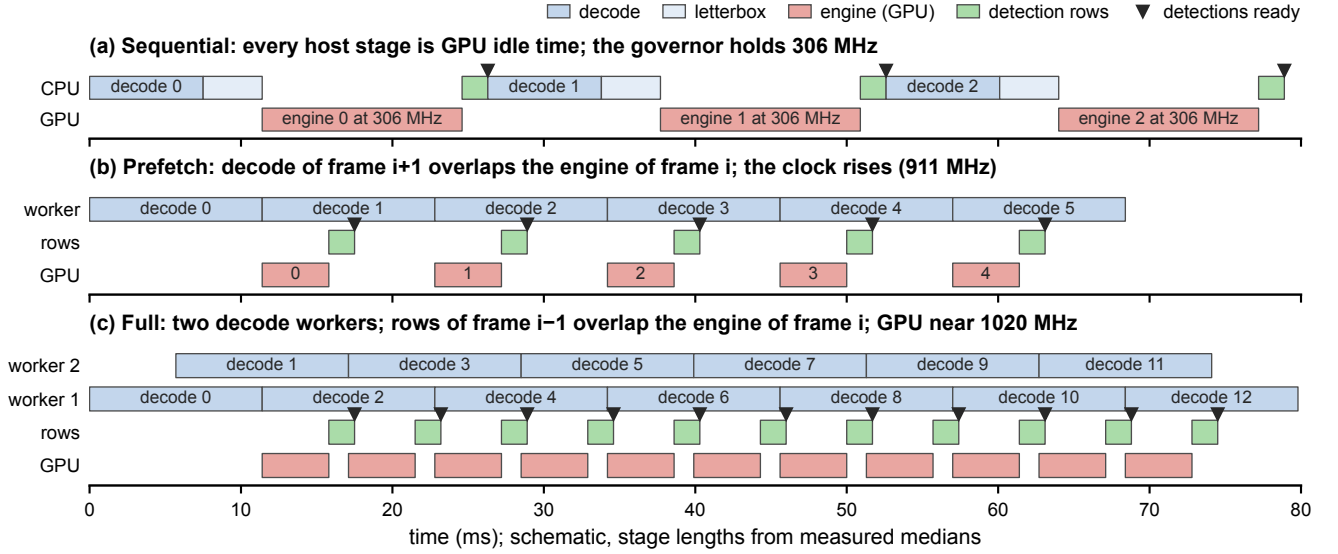


Figure 5: Schematic of the three schedules. The time axis uses the measured stage medians of the sequential float pipeline of Section 5: decode 7.5 ms, letterbox 3.9 ms, rows ≈ 1.5 ms and engine 13.2 ms (host-measured enqueue-to-completion at 306 MHz; the device-side CUDA-event share is 12.5 ms, Table 5). The stage medians sum to 26.0 ms, while the measured period is 29.5 ms (33.9 images/s). The difference is per-image main-thread work outside the timed stages. Markers ($\blacktriangledown$) show when each frame’s detections become ready. The bars show the structure, not the measured throughputs. In (a) every host stage is GPU idle time, so the governor holds the minimum clock. In (b) and (c) the host stages overlap engine execution.

Table 6

Cumulative pipeline ablation for YOLO26n at 640 on all 5,000 COCO val2017 images (batch 1, FP16 TensorRT, MAXN_SUPER). Each cell is the mean of three rotated runs. Run-to-run throughput ranges are below 2.5% of the mean for every cell except two under default governors: the float sequential pipeline (33.1–34.3 images/s) and prefetch with stream synchronization (111.5–119.1 images/s). The $\dagger$ row is a diagnostic, not a cumulative step. It reverts the previous rung’s event synchronization to isolate its effect (Section 6.3). The last two rows are variants of the full uint8 pipeline. AP is official COCOeval. Each row’s predictions were byte-identical across all six runs (both clock settings). Rows with the same input type share one prediction file, except the Ultralytics baseline, which exports its own engine and has its own prediction hash. GPU MHz is the mean sampled devfreq clock under the default governor (always 1020 when locked). CPU ms is process CPU time per image; W is mean module input power (VDD_IN rail). Table headers abbreviate images per second as img/s.

Pipeline	AP	AP _s	Default governors				jetson_clocks			
			img/s	mJ/img	W	GPU MHz	CPU ms	img/s	mJ/img	W
Ultralytics predict	0.4026	0.1978	31.1	194.6	6.06	306	20.5	73.9	139.3	10.29
Ours: float input, sequential	0.4024	0.1975	33.9	187.6	6.36	306	21.9	59.9	165.3	9.90
+ in-graph uint8 preprocessing	0.4026	0.1977	40.1	156.1	6.26	306	17.8	90.1	119.1	10.74
+ event synchronization	0.4026	0.1977	50.0	150.7	7.54	306	21.5	90.4	119.9	10.84
$\dagger$ + decode prefetch (stream sync)	0.4026	0.1977	116.1	83.7	9.72	736	13.5	158.6	86.2	13.67
+ decode prefetch	0.4026	0.1977	146.6	87.2	12.77	911	12.7	159.4	87.2	13.90
+ post-processing overlap	0.4026	0.1977	175.2	81.2	14.23	999	10.6	182.6	80.5	14.69
+ second decode worker	0.4026	0.1977	213.8	74.5	15.94	1019	11.6	217.9	74.5	16.24
Full scheduling, float input	0.4024	0.1975	89.2	109.9	9.80	599	21.6	98.5	122.2	12.03
Full, arrival-aware overlap	0.4026	0.1977	194.8	77.9	15.17	1011	12.8	214.2	75.2	16.12
Full, GC paused in timed loop	0.4026	0.1977	230.2	71.6	16.49	1016	11.2	231.8	72.4	16.79

6. RQ2: Which pipeline decisions matter

6.1. Cumulative ablation on COCO val2017

Fig. 5 illustrates the three schedules. Table 6 and Fig. 6 report the cumulative ladder on all 5,000 val2017 images. Model, engine and accuracy are fixed.

Accuracy. The uint8 pipeline scores 0.40257 AP and 0.19766 AP_s, against 0.40259 and 0.19779 for the Ultralytics predictor on its own engine; the float-input variant scores 0.40240. The ≈ 0.6 -point gap to the 40.9 AP reported for YOLO26n on a desktop GPU with its own export [7] is thus a property of the evaluation stack, not of the pipeline.

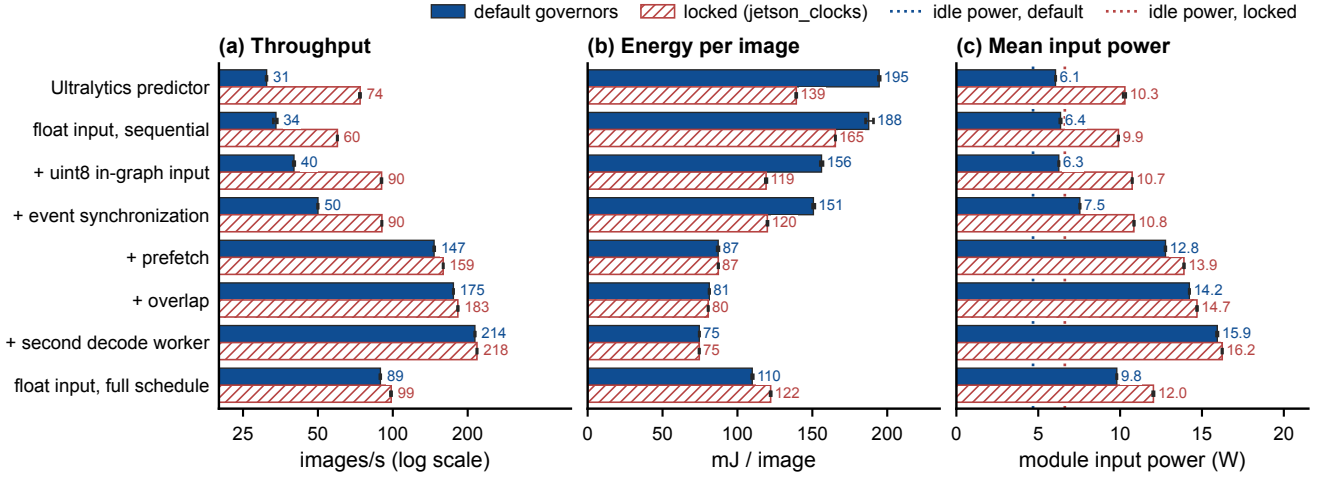


Figure 6: Throughput, energy per image and mean input power along the ladder of Table 6 (bars: mean; whiskers: minimum-maximum of three runs). Dotted lines in (c) mark idle input power (4.68 W default, 6.62 W locked).

Agreement required one decoder fix: an earlier version subtracted the fractional letterbox padding instead of the integer placement offset, which shifted boxes by half a pixel for odd padding and measurably lowered AP on the verification split. Every pipeline was therefore checked with official COCOeval on the full split.

Baselines. Under default governors the Ultralytics predictor and the float-input sequential pipeline reach 31.1 and 33.9 images/s, with the GPU at 306 MHz throughput. Locking clocks with `jetson_clocks`, the usual benchmarking recommendation on Jetson [33], raises them to 73.9 and 59.9 images/s at 3.5–4.2 W more input power.

Host-side changes help a sequential schedule only through the host. Moving normalization and the channel transpose into the engine saves 4.1 ms of CPU time per image and raises throughput by 18% (33.9 to 40.1 images/s). Waiting on an event instead of the stream raises it to 50.0 images/s; equivalently, the stream-synchronized pipeline loses 20%. The GPU clock (306 MHz) and engine time (13.2 ms) are unchanged, so the gain comes from the host side, at a cost of 1.3 W and 3.7 ms of CPU time per image (Section 6.3).

Overlapping CPU and GPU work changes the governor’s regime. Prefetching the next image’s decode and letterbox is the largest single step, from 50.0 to 146.6 images/s. The GPU is now idle only briefly, and its mean clock rises to 911 MHz. The engine’s enqueue-to-completion time falls from 13.2 to 4.4 ms, close to its back-to-back value. Overlapping row construction of image $i-1$ with inference of image i gives 175.2 images/s, and a second decode worker 213.8 images/s. More workers do not help: in a separate sweep over one to five workers (two runs each), throughput was 174.9, 214.2, 215.9, 216.4 and 216.6 images/s, so the third to fifth workers add 1.1% together. With two workers the GPU’s measured load is already 0.90 (0.87 in the load-logging session of Section 5) and its clock is at the maximum, so the pipeline is limited by the engine and the main thread, not by decode. Extra workers only lengthen the queue: median

per-image latency (pipeline entry to finished rows) grows from 17.2 ms with two workers to 30.1 ms with five. The full pipeline is 6.9× faster than the Ultralytics predictor under default governors, but that predictor is a weak baseline. The strongest sequential configuration measured (uint8 input, event synchronization, all clocks locked) reaches 90.4 images/s; the full pipeline under default governors is still 2.4× faster, at 0.4026 AP in both cases.

Locking clocks becomes unnecessary. With locked clocks the full pipeline reaches 217.9 images/s, only 1.9% more than under the default governor, at the same 74.5 mJ per image and 0.3 W more power. Once the pipeline keeps the GPU busy, the stock governor already selects the top clock. Locking speeds up the sequential pipelines by 1.8–2.3× (2.4× for the Ultralytics predictor).

Preprocessing placement interacts with the schedule. The float-input full pipeline reaches only 89.2 images/s. Host-side normalization raises process CPU time from 11.6 to 21.6 ms per image, so the CPU becomes the bottleneck (Eq. (4)) and the GPU clock averages only 599 MHz. In-graph preprocessing is thus worth 18% in a sequential pipeline but 2.4× in the full one. The technique is not new (Ultralytics 8.4.172 already copies uint8 data and normalizes on the GPU); its value depends on whether the schedule keeps the GPU busy.

Where the energy saving comes from. Energy per image falls from 194.6 mJ (Ultralytics) to 74.5 mJ, a factor of 2.6. By Eq. (5), most of this saving comes from amortizing the idle floor (4.68 W) over more images per second. The marginal energy above idle is 44.1 mJ per image for Ultralytics and 52.6 mJ for the full pipeline; across all default-governor rungs it spans 39 to 57 mJ, against 22 to 150 mJ for the idle-floor term. The full pipeline thus does slightly more work above idle (higher clocks, two decoders) but finishes much sooner. Its advantage holds on a backlog or when the device can sleep after finishing, but not necessarily at a fixed camera rate (Section 7).

6.2. Garbage-collection pauses in the evaluation loop

The power traces of the optimized schedules contain short dips (Fig. 4). In the timed window of the full pipeline under the default governor, each run contains six to seven dips of more than 2 W below the median power, with onsets at about 4.1, 6.0, 8.3, 11.3, 14.8 and 19.1 s. The intervals between dips grow from 1.9 to 4.3 s. This pattern matches CPython’s generational garbage collector. A full collection runs only when the number of long-lived objects has grown by 25% since the last one, and every full collection traverses all of them [37]. The evaluation harness, like most COCO evaluation scripts, keeps every detection as a Python dictionary until the end of the run. For val2017 these are 732,651 detections, each a dictionary holding a box list, or about 1.5 million objects tracked by the collector. Each full collection therefore stalls the main thread for longer, the GPU drains, and the power drops.

To confirm the cause, a variant was added that performs one collection before the timed window and then disables the cyclic collector until the window ends. Reference counting still frees all acyclic objects, so memory use is unaffected for this workload. The predictions are byte-identical to the other uint8 rows (Table 6). No run of this variant contains a dip. The standard deviation of the sampled input power falls from 0.88–0.90 W to 0.16–0.20 W under the default governor and from 0.79–0.80 W to 0.07–0.09 W with locked clocks. Throughput rises from 213.8 to 230.2 images/s, so the collector costs 7.1% of the measured throughput. Energy per image falls from 74.5 to 71.6 mJ. With locked clocks the cost is 6.0% (217.9 versus 231.8 images/s).

This gain is not counted as a pipeline improvement. A deployed detector would pass detections on rather than accumulate them, and its collector would have little to scan. The effect matters for benchmarking. An evaluation loop that accumulates results penalizes a fast pipeline more than a slow one, because the same pauses are a larger share of a shorter run. The pauses also arrive at irregular, growing intervals, which inflates tail latency in long runs. All comparisons against the Ultralytics predictor therefore use the rows with the collector enabled, which is the more conservative choice for the proposed pipeline. The rest of the paper follows the same rule.

6.3. Synchronization: spinning versus sleeping

To isolate the 20% effect of the synchronization call, both calls were crossed with the three device scheduling flags [38] on the first 2,000 val2017 images (Table 7). By default, `cudaEventSynchronize` busy-waits, whereas `cudaStreamSynchronize` follows the device scheduling policy, which here lets the host thread yield or sleep.

The effect is spinning versus sleeping. In the sequential schedule, event synchronization gives 49.9–50.0 images/s under all three flags. Stream synchronization gives 40.5 images/s with `auto` and 40.0 with `blocking-sync`, but 50.0 with `spin`. The effect (19.8% on the ladder rungs of Table 6 and 16.7% in the pinned-CPU experiment

below) thus depends only on whether the waiting host thread spins or sleeps, not on events versus streams. On this stack, `cudaEventSynchronize` spun under every device flag, consistent with the CUDA documentation, which ties blocking event waits to the `cudaEventBlockingSync` creation flag [38]; the events used here do not set it. Spinning costs 3.7–4.2 ms of process CPU time per image and 1.2 W of module input power, yet energy per image is 4% lower (151.1 versus 157.2 mJ) because each image finishes sooner.

Engine time is unchanged; host work slows down.

The GPU clock is 306 MHz and the engine’s enqueue-to-completion time is 13.2–13.3 ms in all six sequential cells, so the waking thread does not measurably miss the completion. (The span is not pure GPU time: with the host clamped at 115 MHz it grows to 31.5 ms, as it then contains the host’s launch path.) The difference lies in the host stages. The per-image period grows from 20.0 to 24.7–25.0 ms, and the sampled clock of the fastest CPU core falls from 1728 MHz to about 1643–1694 MHz. The suspected cause is the CPU governor (`schedutil`), which clocks each frequency domain in proportion to the estimated utilization of its CPUs [39]. While the host thread sleeps through the 13 ms engine phase, its core appears idle and the domain is down-clocked, so the decode, letterbox and row stages of the next image run at the lower clock.

A controlled test. Without changing the governor, the sequential pipeline was pinned to CPU 0, and the clock of its frequency domain (CPUs 0–3) was logged at 20 Hz (Table 8). With stream synchronization the domain averaged 62% of its maximum clock. A separate busy-loop process on CPU 1, doing no pipeline work, holds the shared domain at 1728 MHz. Throughput then rose to 49.0 images/s, closing 97% of the gap to event synchronization (49.2 images/s), and the pipeline’s own CPU time fell. The busy loop had no effect on the event-synchronized pipeline, which already held the maximum clock. One confound remains: a busy loop also keeps the cluster out of deep idle states, which shortens wake-up latency. Clamping the whole domain at its 115 MHz minimum, where the governor cannot respond to the loop, bounds this confound. The loop then changes nothing (7.6 versus 7.7 images/s, under 1%, mean of two runs), so its +20% at the shipped governor is a clock effect. The clamp test cannot rule out sub-millisecond idle-state or wake-up effects at operating clocks, because at 115 MHz a microsecond-scale cost is invisible next to 130 ms frames. The `uclamp` experiment below, however, acts on frequency alone and recovers about 87% of the loss, which bounds the idle-state share to at most 13%. A spinning wait keeps the CPU clock high as a side effect.

An unprivileged remedy. A busy loop is a diagnostic, not a fix. Utilization clamping [9] tells `schedutil` directly that a task needs a high clock. A task can raise its own `uclamp.min` with `sched_setattr`, without root privileges on the test system, and threads it creates inherit the clamp. Setting `uclamp.min` of the stream-synchronized pipeline to its maximum (1024), with no other change, raised the domain clock

Table 7

Synchronization primitive $\times$ CUDA device scheduling flag (first 2,000 val2017 images, default governors, mean of two runs, uint8 engine). “Enq. $\rightarrow$ done” is the median enqueue-to-completion wall time of one inference (Table 3); CPU MHz is the mean sampled clock of the fastest core, which is not necessarily the core that runs the feeder thread. Run-to-run throughput spread exceeds 3% only in the two sleeping-wait stream cells (8.6% and 3.5%, from clock wandering between repeats). Energy-per-image spread is below 1.3% throughout.

Mode	Sync	Flag	img/s	mJ	W	GPU MHz	CPU MHz	CPU ms	Enq. $\rightarrow$ done ms
sequential	stream	auto	40.5	157.2	6.37	306	1643	17.3	13.27
		spin	50.0	151.1	7.55	306	1728	21.5	13.20
		blocking	40.0	156.2	6.26	306	1694	17.8	13.30
	event	auto	50.0	150.6	7.52	306	1728	21.4	13.20
		spin	50.0	151.3	7.56	306	1728	21.4	13.21
		blocking	49.9	151.0	7.53	306	1728	21.4	13.24
prefetch	stream	auto	107.4	83.8	9.01	674	1647	14.2	6.25
		spin	140.4	87.2	12.24	860	1727	13.1	4.63
		blocking	110.6	84.1	9.31	694	1642	13.8	6.11
	event	auto	141.5	86.8	12.28	866	1728	13.0	4.62
		spin	140.5	86.7	12.18	862	1728	13.1	4.62
		blocking	140.5	86.8	12.20	862	1728	13.1	4.63

Table 8

CPU-clock test. Sequential uint8 pipeline pinned to CPU 0 (first 2,000 val2017 images, default governors, mean of two runs). “Busy loop” marks a separate process spinning on CPU 1, which shares CPU 0’s frequency domain; “uclamp” sets the pipeline’s `uclamp.min` to 1024 (bottom block, a separate session with its own stream-sync and event-sync references). MHz is the sampled clock of that domain’s cluster; CPU ms counts only the pipeline process. All runs produced identical predictions.

Sync	Remedy	img/s	p50 ms	CPU ms	MHz	GPU ms	W	mJ/img
stream	–	41.0	23.8	15.1	1069	13.29	6.30	153.7
stream	busy loop	49.0	19.9	10.1	1728	13.24	7.69	157.1
event	–	49.2	19.8	20.2	1728	13.21	7.47	152.0
event	busy loop	49.2	19.8	20.2	1728	13.19	7.96	161.7
stream	–	41.3	23.6	14.7	1088	13.29	6.33	153.2
stream	uclamp	48.0	20.3	10.5	1450	13.23	6.81	142.0
event	–	49.0	19.9	20.2	1728	13.23	7.47	152.2

to 1450 MHz and throughput from 41.3 to 48.0 images/s (Table 8, bottom). Against the same-session event-synchronized reference (49.0 images/s), the clamp closes about 87% of the sleeping-wait penalty. The recovery is partial because the clamp acts only while the task is runnable, not while it sleeps through the engine phase. Module input power rose by only 0.48 W, so energy per image fell by 7%, from 153.2 to 142.0 mJ. This is below both the spinning wait (151.1–151.3 mJ) and the busy loop (157 mJ), because the clamp raises the clock without burning a core. Per-task clamping is thus a practical alternative to spinning when a sleeping wait is required.

The two governors compound. With prefetch, sleeping waits reduce throughput from about 141 to 107–111 images/s, a 21–24% loss. The slower host leaves the GPU idle for longer, the GPU governor settles at 674–694 instead of

860–866 MHz, and the engine time rises from 4.6 to 6.1–6.3 ms. A drop in the CPU clock thus lowers the GPU clock as well. This antagonism has been described for the stock governors of the same board family [17]: while the GPU computes, the host looks idle and is down-clocked, so it runs slowly exactly when the GPU next needs data. The present measurements trace the trigger to the CUDA wait policy, bound the effect with two controls, test an unprivileged per-task remedy, and show its propagation to the GPU clock through the duty cycle.

The clamp also lifts the GPU clock. If the compound-ing runs from the host clock to the GPU clock, re-arming the CPU governor should restore both. `uclamp.min` was therefore set to 1024 on the prefetch rung, where the sleeping wait costs the most (Table 9, two repeats per cell). With blocking synchronization, the clamp raises the feeder cores’ clock from 1615–1620 to 1714–1716 MHz. Faster submission raises the GPU’s duty cycle, and the GPU governor settles at 827–860 instead of 631–657 MHz, the range of the spinning and event-synchronized variants. Engine time falls from 6.3–6.9 to 4.7 ms, and throughput rises 34% (101.8–104.2 to 136.3–140.2 images/s) at unchanged energy per image (84.3 to 85.2–86.7 mJ). The clamp thus recovers essentially all of prefetch’s sleeping-wait penalty without burning a core.

On the full pipeline the wait policy is free. The full two-worker schedule is simpler (Table 9, bottom). With the decode pool active, a *sleeping* event wait (per-frame events created with `cudaEventBlockingSync`) reaches 215.5–215.9 images/s, indistinguishable from the spinning reference (215.3–215.6), with 0.3 ms less process CPU time and 0.4–0.5 mJ less energy per image. Locking the clocks changes neither arm (215.1–215.6 sleeping, 214.7–215.5 spinning), and the clamp has nothing left to recover (215.0–215.5). Here the host never goes idle on the critical path. While the main thread sleeps through one completion, two

Table 9

Wait policy and utilization clamping on the optimized pipelines (uint8 engine, first 2,000 val2017 images, mean of two repeats; identical predictions throughout, sha256 prefix df5a48fe35cd). Prefetch rows: blocking stream synchronization, default governors, a separate earlier session from Table 8 with its own same-session references. Full-pipeline rows (this session): every arm runs the decode pool (`-prefetch -overlap -workers 2`); “event, sleeps” uses per-frame events created with `cudaEventBlockingSync`; locked is `jetson_clocks`. *CPU ms* is the process CPU time per image.

Pipeline	wait	clocks	clamp	img/s	CPU ms	mJ/img
prefetch	stream, sleeps	default	–	103.0	14.8	84.3
prefetch	stream, sleeps	default	uclamp	138.3	10.7	86.0
full w2	event, spins	default	–	215.5	10.6	72.9
full w2	event, sleeps	default	–	215.7	10.2	72.4
full w2	event, sleeps	default	uclamp	215.3	10.3	72.7
full w2	event, sleeps	locked	–	215.3	10.3	72.8
full w2	stream, sleeps	default	–	164.8	10.2	83.3
full w2	stream, sleeps	locked	–	164.3	10.2	83.1

decode workers keep the domain busy, so the governor sees no utilization dip. The wait policy matters only where a single thread sleeps through each engine phase: the sequential and prefetch rungs. A GIL control on the same schedule (switch interval shortened tenfold, 5 to 0.5 ms, under locked clocks) also changes nothing: 215.0–215.8 versus 215.1–215.6 images/s.

Stream synchronization costs a quarter through its boundary, not its wait. Blocking *stream* synchronization in the enqueue-ahead loop still loses 24% (163.9–165.1 versus 215.3–215.9 images/s), and the same 24% with locked clocks (163.9–164.7), where the GPU stays at 1020 MHz. The cost thus comes from neither the sleeping wait nor the governor, but from the completion boundary. `cudaStreamSynchronize` waits for *all* work in the stream, including the frame just enqueued, so the rows of frame $i-1$ are finished only after frame i has executed. The loop degenerates to in-order completion with one frame of slack, and the enqueue-to-completion window stretches from 7.2 to 10.0 ms. The per-frame event, spinning or sleeping, preserves the overlap.

Locking clocks removes the governor-mediated penalty where there is one. `jetson_clocks` fixes the CPU clock at its maximum as well as the GPU clock. On the sequential and prefetch schedules, stream and event synchronization then differ by less than 1% (Table 6), so the roughly 20% penalty on these schedules is governor-mediated. On the full two-worker schedule, sleeping and spinning event waits agree within 1% at both clock settings, and only the stream-sync boundary (above) costs throughput, identically at both.

The contrast between schedules therefore depends on the governor. The full two-worker pipeline outperforms the event-synchronized sequential pipeline by 4.28 $\times$ under the default governors (213.8 versus 50.0 images/s, Table 6) but only by 2.41 $\times$ with locked clocks (217.9 versus 90.4).

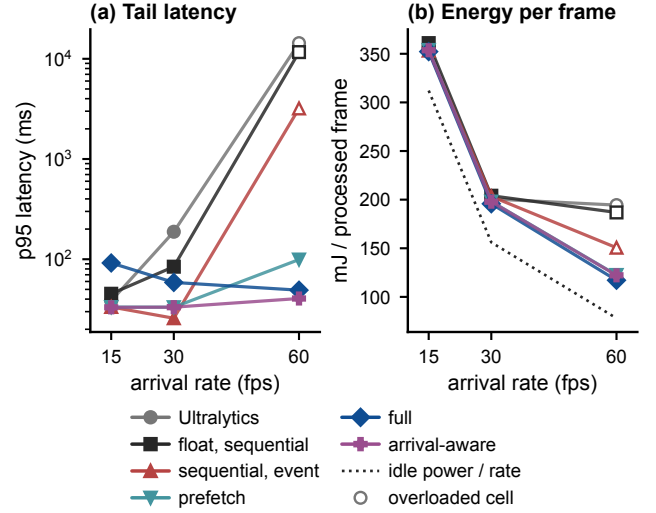


Figure 7: Streaming at 15, 30 and 60 fps under default governors. (a) p95 latency from frame release to finished rows (log scale). Open markers denote overloaded cells (processed rate below arrivals), whose latency reflects a queue growing over the finite run, not a steady state. (b) Module input energy per *processed* frame (the denominator when a pipeline is overloaded). The dotted line is idle power divided by the frame rate.

Locking does not raise the overlapped pipeline’s ceiling; it removes the sequential baseline’s self-inflicted penalty. The reported benefit of scheduling thus depends on the benchmark’s power-management regime.

Answer to RQ2. The decisions that matter change the GPU’s duty cycle: prefetching (2.9 $\times$), overlapping post-processing and adding a decode worker (a further 1.5 $\times$), and how the host waits for the GPU. On the sequential and prefetch schedules the wait penalty is governor-mediated: a sleeping wait lets the CPU governor lower the host’s clock, and through it the GPU’s; locked clocks nearly remove it. On the full two-worker schedule, sleeping and spinning event waits agree within 1% at default and locked clocks, and only stream synchronization in the enqueue-ahead loop still costs 24%, identically at both (Section 6.3). In-graph preprocessing matters once the schedule is no longer sequential. Locking clocks only partly substitutes for a good schedule and adds just 1.9% to one. Finally, garbage-collection pauses in the harness cost the full pipeline 7.1% of its measured throughput.

7. RQ3: Latency and energy at camera rates

A camera delivers frames at a fixed rate rather than as a backlog. The first 1,000 val2017 images were therefore released at 15, 30 and 60 fps, and for each frame the latency from release to finished detection rows and the module input energy were measured (Table 10 and Fig. 7). The arrival-aware variant (Section 4.4) is included; on a backlog it is 8.9% slower than the full pipeline (Table 6).

Table 10

Streaming at fixed arrival rates (first 1,000 val2017 images, YOLO26n 640, default governors, mean of two runs). Latency runs from frame arrival to finished detection rows (ms). Energy is module input energy per frame (mJ). “sat.” marks a pipeline that cannot sustain the arrival rate. Its queue grows without bound, so the achieved rate is given instead of latency. [†]Near-saturated cell: the achieved rate is 29.8 images/s against 30 fps arrivals in both repeats, so the queue drains too slowly to clear between bursts and p95 includes the resulting queuing. Idle input power alone is 312, 156 and 78 mJ per frame at 15, 30 and 60 fps.

Pipeline	15 fps			30 fps			60 fps		
	p50	p95	mJ	p50	p95	mJ	p50	p95	mJ
Ultralytics	32.0	38.4	357	36.2	188.0 [†]	201	sat. (31/s)		194
Float, sequential	39.2	45.2	361	32.2	84.3	204	sat. (34/s)		187
uint8, seq., event	27.4	33.4	353	21.6	25.8	203	sat. (50/s)		151
+ prefetch	26.8	33.3	354	26.7	33.2	198	20.9	99.3	123
+ overlap, 2 workers	85.3	91.8	352	51.9	58.7	196	35.5	49.1	117
+ arrival-aware overlap	27.0	33.1	354	26.5	33.2	198	20.7	40.6	122

Capacity. Pipelines slower than the arrival rate accumulate a growing queue. At 60 fps these are the Ultralytics predictor (31.0 images/s achieved), the float sequential pipeline (34.2) and the uint8 sequential pipeline with event synchronization (49.7; rounded in Table 10). The Ultralytics predictor is also near capacity at 30 fps (29.8 images/s). Short stalls there create slowly draining queues, so its latency is strongly right-skewed (mean 55–64 ms against medians of 36–37 ms). Arrivals are anchored at the end of the harness warm-up; anchoring them earlier would inflate this cell’s p95 from 188 to 447 ms. Dropping the first 100 frames of every run moves the below-capacity p95 values by at most 4 ms (188 to 192 ms for this cell), so the remaining tails are steady-state queue drain, not start-up transients. Backlog capacity thus matters even for a camera: it is the margin that absorbs transient stalls.

Energy is set by the idle floor. At 15 fps every pipeline uses between 352 and 361 mJ per frame, of which the idle floor alone is 312 mJ (86–89%). At 30 fps the spread is 196–204 mJ against a floor of 156 mJ. On a backlog the full pipeline uses 2.6× less energy per image than the Ultralytics predictor, but at 30 fps it uses only 2.5% less (195.7 versus 200.7 mJ). Only at 60 fps, where the slow pipelines saturate, does the schedule decide energy again: 117–123 mJ per processed frame for the pipelines that keep up, against 151–194 mJ for those that do not.

Eq. (6) describes this behavior. With the backlog idle power and marginal energy and the arrival rate substituted, it agrees with every below-capacity streaming cell within −0.2 to +11.6% (mean +4.6%). The two slightly negative cells (the Ultralytics predictor at 15 and 30 fps, −0.2% each) lie inside the repeat spread of the backlog marginal estimate. For the three pipelines that saturate at 60 fps, the arrival-rate substitution underestimates by 10–37%, because a queued pipeline never returns to the idle state the model assumes; substituting the measured processed-frame rate (as the published aggregates do) reduces the error to 0.3–0.5%.

The agreement comes mostly from the idle term, which dominates at camera rates. The marginal term alone is overestimated by 21–39% for the uint8 pipelines below capacity

but agrees within −1.9 to +3.4% for the float-input sequential pipeline and the Ultralytics predictor (which normalizes on the GPU, Section 6). The backlog-to-stream clock gap does not explain the uint8 discrepancy: the worst cell (the event-synchronized sequential pipeline at 15 fps, +39%) runs at 306 MHz in both. The residue cannot be attributed from these aggregates, since CPU sleep states and any inter-frame GPU power gating differ between the workloads and the module rail does not separate them. The backlog marginal energy is not a prediction of the streaming value but an empirical upper envelope for it in every measured uint8 configuration, and lies within −2 to +4% of it for the other two pipelines. At 15 and 30 fps the GPU runs at about 306 MHz in all pipelines, against about 1 GHz for the optimized pipelines on the backlog, so marginal energy per frame is lower: the governor does what it is designed to do.

Locking clocks with `jetson_clocks` raises the idle floor from 4.68 to 6.62 W, which at 30 fps is 221 mJ per frame before any work, already 8–13% more than the total energy of every default-governor pipeline in Table 10.

Latency is set by the schedule. At low rates the overlapped schedule, despite the highest backlog throughput, has the worst latency: a median of 85.3 ms at 15 fps, against 27.4 ms for the sequential pipeline. Overlap finishes the rows of frame i only after frame $i+1$ has been decoded and enqueued, so every result waits about one inter-arrival period (66.7 ms at 15 fps), a penalty that shrinks as the rate grows (Table 10). The sequential pipeline is fastest per frame at 30 fps (median 21.6, p95 25.8 ms), where its backlog capacity of ≈ 50 images/s exceeds the arrival rate by two thirds, but it fails at 60 fps. Plain prefetch keeps up at 60 fps, but its p95 rises to 99.3 ms: the governor runs it at about 410 MHz, so its margin over the arrival rate is small and transient stalls build queues.

The sequential pipeline’s per-frame cost also depends on the arrival rate: for the same work, its median latency (27.4 versus 21.6 ms) and CPU time per frame (29.4 versus 23.5 ms) are both higher at 15 than at 30 fps. The logged clock of the fastest core (1705 MHz at 15 fps, 1682 MHz at 30 fps) does not explain this. A per-core effect on the waiting thread’s core remains possible, but the measured clocks do

not support the down-clocking mechanism of Section 6.3 here; the extra CPU time is at least as consistent with per-frame sleep/wake overhead.

The arrival-aware variant combines the two behaviors, finishing the in-flight frame when no next frame is available and overlapping when frames queue up. Its p95 latency is within 7.4 ms of the best pipeline at every rate, it is the only pipeline whose p95 stays at or below 41 ms at all three rates (Table 10), and its energy is within 0.4–4.3% of the full pipeline. Its predictions are byte-identical to those of the other uint8 pipelines.

Answer to RQ3. At camera rates below a pipeline’s capacity, energy per frame is dominated by the idle floor: the tested pipelines differ by at most 2.5% at 15 fps and 4.2% at 30 fps. The schedule mainly determines latency and capacity margin: the one with the best backlog throughput has the worst latency at 15 fps because it holds every result for one frame period. An arrival-aware schedule removes this penalty at a cost of 8.9% of backlog throughput and at most 4.3% of energy. Locking clocks raises the idle floor and is counterproductive in this regime.

8. RQ4: Model-level decisions under a fixed pipeline

8.1. Compression screening and a resolution router: negative results

Before the pipeline study, most of the effort went into the model: L1 and gradient-based channel pruning, mixed-precision QAT, distillation, and a per-image router that chooses between a 512 and a 640 engine from pre-inference statistics. Under a common short recovery budget (1,024 development images, five epochs, one seed), none of the compressed variants reduced compiled engine time at acceptable accuracy. The router looked favorable on its fitting split but lost AP and small-object recall on an independent hash-verified verification split, so it was rejected. Appendix A reports both campaigns in full. These results do not show that compression cannot work. They show that, for a 2.6 M-parameter NMS-free detector on this device and budget, the achievable engine-time savings are a small fraction of the 3× governor effect of Section 5. The rest of this section therefore varies resolution and model size while holding the optimization budget fixed.

8.2. The value of a lower input resolution depends on the schedule

Lowering the input resolution is the cheapest model-level saving. It needs no training, only a new engine. Fig. 8 compares static 512, 576 and 640 engines of the same pre-trained YOLO26n checkpoint on all of val2017, in the naive pipeline (float input, sequential, stream sync) and in the full pipeline. Accuracy falls from 0.4026 AP (640) to 0.3864 (576) and 0.3748 (512), and AP_S from 0.198 to 0.175 and 0.160. In the naive pipeline, 512 buys 27% more throughput (34.3 to 43.4 images/s) and 22% less energy per image. In the full pipeline the gain shrinks to a few percent: 214.6,

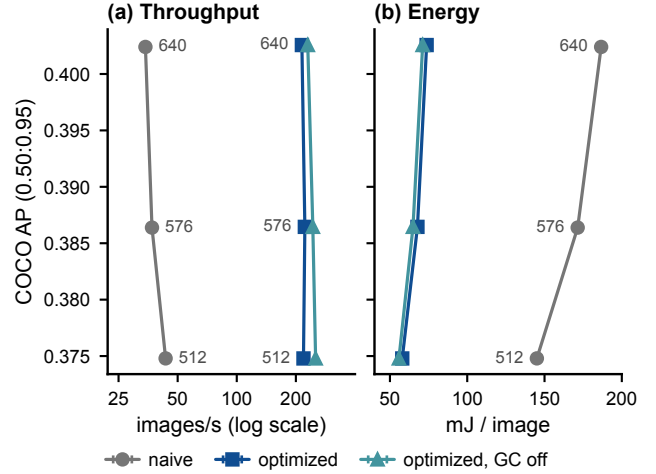


Figure 8: COCO val2017 AP against throughput (images/s, log axis, so equal distances are equal ratios) and energy per image (mJ) for static 512/576/640 YOLO26n engines, default governors, mean of two runs (whiskers: repeat min/max; labels: input size). Circles: naive float sequential pipeline. In the full pipeline (squares), lower resolution buys only 2–4% throughput because the pipeline is limited by the main thread, not by GPU throughput; the GPU already runs at its maximum clock. Disabling CPython’s cyclic GC (triangles, identical predictions) raises all three points and widens the spread to 6–10%.

222.2 and 218.7 images/s for 640, 576 and 512 (+3.5% and +1.9%). The repeat ranges of the lower resolutions lie above the 640 range, so the gain is small but consistent across the two repeats. The pipeline is now limited by the main thread (Section 6). At 1020 MHz the engine’s 4.2 ms sits just below the ≈ 4.7 ms per-image period imposed by the host stages, so the unchanged per-image host work absorbs most of the engine time saved by a smaller input.

Part of that saturation is Python. With CPython’s cyclic garbage collector disabled (`gc.disable()`, identical predictions), the three throughputs rise to 229.9, 244.0 and 252.1 images/s, and the resolution spread widens to +6.1% (576) and +9.7% (512). Cyclic-GC pauses on the main thread thus absorb a further share of the saved engine time. Without them, a lower input resolution still buys roughly a third of what it buys in the naive pipeline, which is not negligible. The remaining host work, the main thread’s per-image period (Section 6), depends on the source image and the pipeline, not on the network input. The energy benefit holds either way, because the GPU finishes each inference sooner: 73.7 to 58.0 mJ per image (21%) on a backlog with the default collector, and 71.2 to 55.8 mJ (22%) with it disabled. The same 2.8 AP points therefore buy a large throughput gain in a naive pipeline and a much smaller one (2–10%, depending on host-runtime hygiene) in an optimized pipeline. A compression or resolution decision evaluated in a naive pipeline, or in `trtexec`, does not carry over to an optimized one.

Table 11

Other NMS-free detectors on all 5,000 val2017 images, 640, batch 1 (default: mean of two runs; locked: one run). “Naive” is float input with sequential execution and stream synchronization. “Full” is uint8 input, two prefetch workers, overlap and event synchronization. MHz is the mean sampled GPU clock under the default governor.

Model	Pipeline	AP	AP _s	Default				Locked	
				img/s	mJ	W	MHz	img/s	mJ
YOLO26s	Ultralytics	0.4769	0.3020	25.1	263.5	6.62	306	63.0	194.4
	Ours, naive	0.4770	0.3019	24.5	271.6	6.66	306	52.2	221.8
	Ours, full	0.4769	0.3019	140.1	131.8	18.47	1019	140.4	133.3
YOLOv10n	Ultralytics	0.3836	0.1901	30.8	198.4	6.12	306	73.0	144.5
	Ours, naive	0.3837	0.1900	33.5	189.4	6.34	306	60.0	167.4
	Ours, full	0.3839	0.1901	212.4	76.9	16.33	1018	215.2	77.8

8.3. Generality: YOLO26s and YOLOv10n

Table 11 repeats the three main configurations for YOLO26s (10.0M parameters) and YOLOv10n. In every case the three pipelines agree in AP to within 4×10^{-4} , and the naive pipelines run the GPU at 306 MHz under default governors. YOLOv10n behaves like YOLO26n: 30.8 images/s with Ultralytics and 212.4 with the full pipeline (6.9 $\times$), at 76.9 instead of 198.4 mJ per image. YOLO26s has 3.9 $\times$ the parameters of YOLO26n and a 1.6 $\times$ longer engine time (6.78 versus 4.22 ms back-to-back). Its full pipeline is limited by the GPU rather than the CPU. It reaches 140.1 images/s (5.6 $\times$ Ultralytics), 95% of the 146.9 inferences/s that trtexec achieves back-to-back, and locking clocks does not change it (140.4). Under the default governors, the full YOLO26s pipeline outruns the Ultralytics YOLO26n predictor with locked clocks (140.1 vs. 73.9 images/s) at 0.074 higher AP and 7.5 mJ less per image (131.8 vs. 139.3). This comparison changes the pipeline and the model at once, so it does not show that the larger model is the better choice; YOLO26n in the same full pipeline is faster still. It shows that the larger model costs less than a naive pipeline, even one running with locked clocks. A deployment decision that compares model sizes in a naive pipeline therefore mostly measures the pipeline.

Answer to RQ4. Once the pipeline is fixed, model-level savings matter much less than the engine benchmark suggests. A 2.8-point AP sacrifice for a lower resolution yields 27% more throughput in a naive pipeline but only 2–4% in an optimized one (6–10% with cyclic GC disabled), plus lower energy on a backlog. The screened compression variants did not reduce compiled engine time at acceptable accuracy within the budget. The router saved at most one eleventh of what scheduling saved, and it failed held-out verification. Whether the schedule effects carry over to other detectors and accelerators is **RQ5** (Section 9). Within this section, YOLOv10n and YOLO26s show speedups of 5.6–6.9 $\times$ over the Ultralytics predictor. For YOLO26s, whose engine is 1.6 $\times$ slower than YOLO26n’s, the choice of pipeline matters more for throughput than the choice between the two model sizes.

9. RQ5: Cross-platform replication

The Jetson study traces the in-app cost to an accelerator governor that reacts to the host pipeline’s duty cycle, coupled to a CPU governor that reacts to how the host waits. To test whether this is board-specific, the core experiments were repeated on two boards with different accelerator architectures (Table 12): a Raspberry Pi 5 with a Hailo-8, a fixed-function INT8 NPU with on-chip NMS and no user-visible frequency control, and a Rubik Pi 3 (Qualcomm QCM6490), measured on its CPU under a three-cluster schedutil DVFS and on its Hexagon HTP NPU through a separately obtained QNN runtime. Both follow the protocol of Section 4: official COCOeval on val2017 (all 5,000 images for accuracy; the ladder uses the first 1,000 on the slower Rubik), the same letterbox geometry and row conversion, hashed predictions, and a rotated two-repeat campaign. Logs are sampled at 20 Hz on the Jetson, 10 Hz on the Pi (the rate its PMIC sustains) and 13–18 Hz on the Rubik, which logs clocks and thermals only. The pinned-frequency probes and the HTP streaming cells are single runs; both are flagged where they appear.

9.1. Raspberry Pi 5 with a Hailo-8 accelerator

The Hailo model-zoo YOLOv8n HEF offloads the whole network, including NMS, to the NPU; the host decodes, letterboxes into a uint8 NHWC buffer and converts the NMS output to COCO rows. The vendor benchmark (hailortcli benchmark) reports 431.6 images/s in throughput mode under both CPU governors, while a conventional sequential application on the same HEF reaches 15% of that (Table 13). The NPU clock is neither user-controllable nor exposed by any register, so an internal NPU DVFS can be neither ruled in nor ruled out.

The ladder mechanisms transfer (Table 13). Prefetching decode gives 1.86 $\times$; overlapping post-processing with a second decode worker gives another 1.5 $\times$; a third worker adds nothing, as on the Jetson. The restructuring reaches 2.7 $\times$ the sequential application (3.3 $\times$ with the spinning wait). Predictions are byte-identical in every rung, repeat and governor (sha256 prefix 455ea22b07f1; official AP 0.320, AP_s 0.122 on all 5,000 images).

Table 12

Replication platforms. The Rubik Pi 3 is measured with two inference stacks: its CPU, and its Hexagon NPU.

	Raspberry Pi 5 + Hailo-8	Rubik Pi 3 (CPU)	Rubik Pi 3 (NPU)
SoC	BCM2712	QCM6490	QCM6490
CPU	4x A76, one domain, 1.5–2.4 GHz	8 cores, three domains, 0.8–2.7 GHz	(same)
Accelerator	Hailo-8 NPU (INT8), on-chip NMS	none used	Hexagon HTP V68 (INT8)
Model	YOLOv8n HEF	YOLO26n ONNX (1,300,6)	YOLO26n QNN ctx, (1,84,8400), host NMS
Runtime	HailoRT 4.23.0	ONNX Runtime 1.30.0	qai-appbuilder 2.50.40 ^b
CPU governor	ondemand, performance	schedutil, performance	
Power sensor	PMIC rails, SoC side only ^a	none usable ^c	

^aExcludes the 5V loads: the Hailo-8, fan and USB.^bPyPI wheel bundling libQnnHtp + V68 skel; the board’s QNN 2.46 predates the QAIRT 2.49.40-built context binary.^cThe board’s battery manager reports inconsistent values; only clocks and thermals are logged.

The wait-policy effect also appears, through a different mechanism. Replacing the sleeping wait on the asynchronous inference handle with a spinning wait gains 22% under ondemand and 29% under performance. Under performance the CPU clock is pinned at 2.4 GHz, so the CPU governor cannot mediate the gain; it sits inside HailoRT’s wait and callback path, which was not profiled further. A separate, smaller governor effect exists: a controlled probe (seq versus seq with a busy spinner) shows the governor holding 2.4 GHz instead of 2.2 GHz, worth 7.7% of throughput at 42% more energy. The NPU time from submission to completion stays at 6.3–9.3 ms across the ladder; the wait inside that interval absorbs the schedule differences.

Energy per image falls 1.8× from sequential to the best variant. These figures cover only the SoC rails (Table 12); the unmeasured NPU energy can itself move with the schedule, so they do not establish whole-system rankings. SoC-side comparisons remain meaningful between configurations sharing the same engine and accuracy; cross-engine comparisons on this board need module-level power measurement. Streaming at 15, 30 and 60 fps is consistent with the Jetson’s idle-floor pattern (Table 17): sequential and overlapped pipelines differ by 9% in energy per frame at 15 fps and converge at 60 fps. The 15 fps gap is indicative only, because its overlapped cell has a 19% two-repeat spread (101–121 mJ) and the sequential pipeline already sits at the idle floor within measurement error (single 60 s idle window: 1.534 W ⇒ 102.3 mJ per frame at 15 fps).

Two differences from the Jetson matter. First, accuracy is not comparable across the boards. The HEF’s on-chip NMS is compiled with a score threshold of 0.2, truncating the low-score tail that COCO AP rewards: AP is 0.320, against the 0.373 that Ultralytics reports for the same network at the conventional 0.001 threshold [40]. The threshold explains the direction and plausibly most of the magnitude; without pre-threshold rows, smaller contributions from the HEF toolchain cannot be ruled out. Because every rung

Table 13

Pipeline ladder on the Pi 5 + Hailo-8 (5,000 val2017 images, mean of two rotated repeats; p50 is the median per-image end-to-end pipeline latency; power is SoC-side only).

Governor	Rung	img/s	p50 ms	SoC mJ/img	CPU MHz
ondemand	seq	65.7	15.1	34.2	2020
	prefetch	122.0	16.1	25.5	2243
	full w1	149.8	14.6	23.0	2377
	full w2	177.9	25.2	25.2	2319
	full w2 spin	217.5	21.4	19.4	2300
	full w3	174.7	31.5	25.2	2327
performance	seq	74.6	13.3	34.8	2400
	prefetch	120.5	15.1	26.1	2400
	full w1	140.7	14.7	22.8	2400
	full w2	169.9	25.5	26.3	2400
	full w2 spin	219.8	21.2	20.0	2400
	full w3	166.4	32.1	27.2	2400

produces byte-identical predictions, this cancels in within-platform comparisons. Second, the default ondemand governor costs less than nvhost_podgov on the Jetson: it sits at 2.0 GHz even in the sequential pipeline and, once the pipeline is overlapped, nearly matches performance (219.8 versus 217.5 images/s at the best rung). The Jetson’s three-fold governor effect is specific to its aggressive GPU governor; what generalizes is that the benchmark number describes a duty cycle the application creates, not a property of the accelerator. One anomaly is unexplained by clocks: on the overlap rungs ondemand is 5–6% faster than performance, while with the spinning wait the order inverts (Table 13); NPU-side power states, not pursued here, may differ.

9.2. Rubik Pi 3: inference on the CPU

The Rubik Pi 3 runs the Jetson float engine’s YOLO26n ONNX graph on the CPU (ONNX Runtime, default threads), so inference competes with the host stages and the wait policy for the same resource. In the overlap rungs each decode worker runs the session call, so up to `max_workers` inferences execute concurrently and the *infer* column includes their

contention. The frequency probes pin the run to the prime core (taskset, intra-op parallelism 1) to exercise a single frequency domain; the ladder runs are unpinned. On all 5,000 val2017 images the board reproduces the Jetson engine’s AP almost exactly (0.4027 versus 0.40257, AP_S 0.1977 versus 0.19766). Multithreaded inference emits rows in a nondeterministic order, so predictions are compared after canonicalization (sorting rows by image, category, score and box): all six rungs produce the same 143,362 canonical rows on the 1,000-image subset in every repeat and governor (24 of 24 cells), and seq, full_w2 and full_w2_spin give identical AP (0.425).

Every effect that dominated on the accelerator boards shrinks or inverts (Table 14). Prefetch alone no longer helps (−5%), because the decode worker steals a core from inference. Overlap with one worker is best, but gains only +28% and nearly triples the median latency, because the queued frame waits behind a 400 ms inference. The spinning wait, worth 20–22% on both accelerator boards, gains nothing, since the spinner competes with the inference threads for cores. The two governors differ by less than the drift between identical repeats, consistent with both sitting at the thermal limit. The SoC reached 95.4–95.8 °C, the firmware ceiling, in every ladder run, and a late repeat is up to 17% slower than an earlier identical one; the Jetson’s median run-to-run range was 0.8%. No hardware throttle counter was read, so the drift is attributed to thermal management by its signature (sustained ceiling temperature plus progressive slowdown), not by direct observation.

What remains controllable is the clock. Pinning the big core’s frequency domain to 806, 1862 and 2707 MHz moves the sequential pipeline through 1.7, 3.1 and 3.8 images/s, and inference time from 575.7 to 255.2 ms, a 2.26× ratio for a 3.36× frequency ratio. Each pin is a single run and the other two domains float, so this is a dose–response, not a scaling law. Frequency is a direct throughput control here, because the pipeline never idles the processor long enough to confuse schedutil. In streaming (2, 5 and 10 fps) both pipelines keep up only at 2 fps, where the sequential variant again wins on latency (p50 290 versus 356 ms) and the big core drops to 2.3–2.5 GHz in the idle gaps. At 5 and 10 fps the queue grows linearly: within a 300-frame stream the sequential median latency reaches 23.9 and 39.3 s, and the overlapped pipeline’s higher capacity (3.8 versus 2.8 images/s) holds it to 8.6 and 26.2 s at a saturated 2.7 GHz. The sequential capacity is below the ladder’s 3.26 because the streaming phase ran late, when the board was hottest. No schedule rescues a pipeline whose capacity is below the arrival rate; here the clock, or a cheaper model, is the only lever.

9.3. Rubik Pi 3 on its Hexagon NPU

The QCM6490’s intended accelerator is a Hexagon HTP (V68) NPU. The board’s QNN runtime predates the available INT8 context binary of the same YOLO26n graph (Table 12), and the newer SDK requires a vendor account. The qai-appbuilder PyPI wheel bundles a matching libQnnHtp and V68 skel; with the DSP (digital signal processor) library

Table 14

Pipeline ladder on the Rubik Pi 3 (CPU inference, 1,000 val2017 images of which 980 are timed after warm-up). Values are the mean of two rotated repeats with the repeat range in parentheses; two repeats cannot support an interval. p50 is the median per-image end-to-end latency and *infer* the ONNX Runtime call. In the overlap rungs the decode workers share the cores that inference uses, so the call itself lengthens. No power sensor is available on this board.

Governor	Rung	img/s	p50 ms	infer ms
schedutil	seq	3.26 (2.96–3.56)	304	254
	prefetch	3.11 (2.91–3.31)	641	269
	full w1	4.17 (3.99–4.36)	896	415
	full w2	3.85 (3.78–3.92)	1212	446
	full w2 spin	3.84 (3.69–4.00)	1237	451
	full w3	4.04 (3.92–4.17)	1415	426
performance	seq	2.97 (2.89–3.05)	329	275
	prefetch	2.96 (2.86–3.06)	671	281
	full w1	4.07 (3.89–4.26)	913	424
	full w2	4.00 (3.88–4.13)	1185	429
	full w2 spin	3.75 (3.63–3.87)	1262	461
	full w3	3.97 (3.84–4.10)	1443	434

path pointed at the wheel, the context runs unchanged. The context, not the pipeline, brings two protocol differences: it is INT8, and it ends in the standard (1,84,8400) head, so NMS runs on the host with the reference val settings (confidence ≥ 0.001 , IoU 0.7, at most 300 detections). It scores AP 0.364 (AP_S 0.175) on all 5,000 images against 0.4027 (0.198) for the float CPU stack, trading 3.9 points of AP for eleven times the throughput of the best CPU pipeline under the same governor (43.6 versus 4.07 images/s under performance; 6.7× under schedutil).

One caveat applies to that comparison. The context’s uint8 score encoding has a quantization scale of 0.01045 (measured on the output grid), so the effective score floor is ≈ 0.01 rather than 0.001: the Hailo mechanism, twenty-fold milder. It truncates only the far tail; on fewer than ten of the 5,000 images the stack emits no rows, all of them images whose float-stack detections score below 0.02. A sensitivity check quantizes the float stack’s scores to 0.01 steps and drops the sub-0.01 tail, which lowers its AP from 0.4026 to 0.3999 (−0.27 points, AP_S −0.5). The score floor therefore explains about 7% of the 3.9-point gap. The check covers only the score floor, not the quantization of boxes or features or the stacks’ different error distributions, so it does not apportion the remaining gap. Within the NPU stack, predictions are byte-identical across every rung, repeat and governor (sha256 prefix 09de71d67bad on the ladder subset), and the accuracy run’s rows for those images match row for row. Each accuracy row is a single run; COCOeval is deterministic, so a repeat could differ only through runtime nondeterminism.

The bundled appbuilder/QNN userspace wedged the DSP session under concurrent multi-context inference, so the pipeline holds exactly one context and issues every NPU call from a single thread; overlap comes from pipelining decode, input fill and host NMS *around* the serialized NPU

calls. Under that discipline no call failed. Separately, an early evaluation-script bug inflated every timed call by a per-call error print; it was fixed, the entire HTP campaign was rerun, and only corrected numbers are reported. Appendix B details both issues and why the bug mattered most where clock sensitivity is examined.

Table 15 shows a return to the accelerator pattern. With a 24–28 ms NPU call, the host stages are again the tax, and the ladder recovers them: +24% under `schedutil` (22.9 to 28.5 images/s, the spinning-wait diagnostic; the best non-spinning rung reaches 27.8) and +52% under performance (28.8 to 43.6 images/s). Under performance the best rung saturates the accelerator: its 22.9 ms period matches the loaded per-call service time, marginally below the 24.4 ms isolated-call median. That median implies about 41 calls/s, which the best rung exceeds, so it overstates the loaded service time. Under sustained submission either queued calls overlap driver-side work or the loaded dispatch share shrinks; the two cannot be separated.

Because NPU calls are serialized, a second decode worker does not pay off. Under performance `full_w2` is slower than `full_w1` and adds 39 ms of p50 latency, since the extra worker only lengthens the queue; under `schedutil` the two tie, because the slowed feeder core binds either way. The governor, irrelevant on the CPU board, is the largest single term here. Under `schedutil` the core that feeds the NPU settles at 0.95–1.0 GHz in the overlapped rungs and at 1.78 GHz in the sequential one: the less host work a rung leaves on the critical path, the lower the clock the governor chooses. The same pipeline runs up to 57% faster under performance (43.6 versus 27.8 images/s on the matched `full_w1` rung; 44% on the matched `full_w2` rung, 39.9 versus 27.7 images/s).

The pinned-frequency probe shows why the host clock matters for an “NPU” call. At 806, 1862 and 2707 MHz the sequential pipeline delivers 16.7, 24.0 and 28.6 images/s, and the measured NPU time falls from 35.8 to 24.4 ms. A least-squares fit decomposes the three points into a fixed ≈ 20 ms NPU kernel plus a host-side dispatch term inversely proportional to the pinned core’s clock, so a large share of each call is remote-procedure-call (RPC) work on the CPU. The decomposition is descriptive, not a validated model: the mid point sits 0.8 ms above the fit, and two parameters over three single-run pins leave one residual degree of freedom. With no observable NPU clock or load register, it also cannot distinguish a clock-invariant kernel from an accelerator DVFS absorbed by the constant term. The governor invariance below is therefore a hypothesis consistent with the data, not a measurement.

The spinning wait gains nothing here (38.2 versus 39.9 images/s under performance): a single serial caller has no asynchronous-handle wait for the spinner to heat. This cell also shows the campaign’s widest repeat spread (34.9–41.5 images/s), which is thermal drift, with the SoC climbing from 83 to 95 °C, not noise. Within the decomposition, what moves with the governor is the call’s host-dispatch share, not the accelerator clock: the sequential

call takes 24.4 ms under performance and 28.3 ms under `schedutil`, while the fitted kernel is governor-invariant. Under the shipped governor the serialized call path at the best rung delivers one completion every 35.1 ms, about 24% slower than the 28.3 ms sequential-call rate, even when the queue never empties. The *infer* column rises to ≈ 47 ms in the overlap rungs because it then includes the queue wait behind the serialized caller, not because the accelerator slows.

Streaming behaves as the capacities predict (Table 17; single runs, so the governor contrast is paired but not replicated). Under `schedutil` the pipeline keeps up at 15 fps (p50 108 ms overlapped, 54 ms sequential) and queues linearly at 30 and 60 fps (p50 0.8 and 5.1 s overlapped, 2.4 and 6.7 s sequential). Under performance the overlapped pipeline sustains 30 fps at p50 47 ms, while the sequential one (capacity 28.8 images/s) already falls behind (p50 0.39 s); at 60 fps both queue (p50 2.1 versus 4.7 s). Predictions agree row for row with the ladder on the shared images.

Baking selection into the graph The Jetson and Rubik CPU stacks fuse selection into the NMS-free (1,300,6) head, and the Hailo HEF runs NMS on-chip, so an end-to-end context was also built for the HTP with the matching QAIRT 2.50.40 toolchain (INT8, 500 calibration images, context binary generated on the board). One silent pitfall arose: the exported head routes the box branch (range $\approx [-50, 800]$) and the score branch ($[0, 1]$) through a single concat–transpose–split, and per-tensor INT8 quantization gives the shared tensor one scale, which zeroes the score channel at runtime. Graph surgery, not calibration, fixes this: the branches are transposed separately, and boxes, scores and classes are exposed as three output tensors with their own scales. Mixed precision is not an option, because the V68 HTP rejects fp16 graphs outright.

The repaired context detects correctly and is faster end to end, because selection moves onto the DSP and host NMS disappears: on this toolchain it runs 45% faster at peak (Table 16), and under performance the overlapped 30 fps stream cuts its p50 latency by 42% (27 versus 47 ms). The same descriptive decomposition puts the fixed NPU kernel at ≈ 11.6 ms, against ≈ 20 ms for the standard head (endpoint and least-squares fits span 11.5–11.6 ms, with the mid probe point 0.3 ms above the fit). The fused head scores 7 AP points lower, with the loss concentrated in small objects, where the in-graph INT8 box-decode chain (regression logits at 0.75 per step, decoded boxes at 3.3 px per step) hurts most. The standard head with host NMS therefore remains the primary HTP stack. Because the two contexts differ in QAIRT release and calibration set, this is a configuration comparison, not an isolated cost of fusing selection.

9.4. What transfers, and what does not

The *benchmark-versus-application gap* appears on all four stacks: 15% of the benchmark on the Pi 5, 4.2 versus 13.2 ms on the Jetson, a 254 ms inference inside a 304 ms loop on the Rubik CPU, and a 28 ms NPU call inside a 43 ms

Table 15

Pipeline ladder on the Rubik Pi 3 with the Hexagon HTP NPU (INT8 context, host NMS; 1,000 val2017 images of which 980 are timed). Values are the mean of two rotated repeats with the repeat range in parentheses; two repeats cannot support an interval. NPU calls are serialized on one thread (see text). *infer* is submission-to-completion of the NPU call. In the overlap rungs it spans the queue wait behind the single serialized caller, not pure accelerator time; the sequential rung gives the pure call (24.4 ms under performance).

Governor	Rung	img/s	p50 ms	infer ms
schedutil	seq	22.94 (22.82–23.06)	43	28.3
	prefetch	25.60 (25.52–25.68)	77	28.8
	full w1	27.75 (27.75–27.76)	133	47.0
	full w2	27.68 (27.65–27.71)	173	46.2
	full w2 spin	28.45 (28.26–28.64)	173	53.2
	full w3	27.70 (27.63–27.77)	209	46.7
performance	seq	28.75 (28.67–28.83)	35	24.4
	prefetch	31.86 (31.74–31.98)	62	24.4
	full w1	43.59 (43.56–43.61)	89	40.0
	full w2	39.89 (39.54–40.24)	128	45.3
	full w2 spin	38.22 (34.92–41.53)	135	47.2
	full w3	40.01 (37.47–42.56)	146	42.5

loop on the HTP under the shipped governor. What fills the gap differs by stack, as summarized at the end of this section.

The *host-pipeline ladder* helps on all four stacks, by an amount that depends on what else moves with the schedule (best rung versus sequential, shipped governor). On the Jetson it reaches 6.3× against the naive sequential rung (float input, stream synchronization) and 4.3–5.3× against the stronger sequential variants of Table 6, because overlap also lifts the GPU out of its DVFS floor; the 2.4× headline of Section 1 uses the locked-clock baseline. The gain is 3.3× on the Pi, the most host-bound of the four, and 1.3× on the Rubik CPU. On the HTP it is only 1.2×, because schedutil settles the feeder core near 1 GHz once the pipeline overlaps; fixing the governor first re-opens it to 1.5× (Table 15).

The *wait-policy effect* appears on both asynchronous-handle accelerators, through different mechanisms. On the Jetson the spinning wait gains 20–25% on the sequential and prefetch rungs, mediated by the CPU governor (Section 6.3), and nothing on the full two-worker pipeline, whose host never idles on the critical path. On the Pi it gains 22–29% on the *best* rung and persists at a pinned CPU clock, so it lives in the runtime’s wait path. It vanishes where no such wait exists: on the Rubik CPU the spinner steals cycles from inference, and the HTP’s single serial caller waits on a bounded queue rather than an async handle.

The *accelerator governor*, the Jetson’s dominant term, is observable only where the accelerator exposes a clock. Neither NPU does; an internal DVFS cannot be ruled out, but none is needed to explain the measurements. The measured per-call NPU time includes host-side wait and runtime queueing (it grows from 6.3 to 9.3 ms across the Pi’s ladder; the HTP’s infer column spans queue waits), and once that share is accounted for, neither stack shows a residual that moves with the host duty cycle. The governor story moves

rather than disappears: on the HTP the CPU governor is the largest term (57% on the matched full_w1 rungs), because schedutil down-clocks the feeder core and each call’s host-dispatch share grows as that clock falls (Section 9.3).

Thermal limits bind on the Rubik CPU stack, which sits at the 95 °C ceiling with measurable repeat drift. The NPU stacks approach the same ceiling under performance (91–95 °C). The Pi shows no drift; the two heaviest performance overlap rungs on the HTP do (full_w2_spin and full_w3, which falls from 42.6 to 37.5 images/s between repeats), so these two cells mix governor and thermal effects; the schedutil rows and both sequential ladders show no drift.

Streaming follows capacity on the NPU boards as Section 7 predicts (Table 17): the Pi keeps up at every rate, since its sequential capacity already exceeds 60 fps, and the HTP queues precisely when the arrival rate passes the ladder capacity. The Jetson’s plain-overlap hold of each frame’s rows for about one arrival period (Section 7) belongs to the harness’s double-buffered host loop and does not occur on either NPU stack. HailoRT delivers results through an asynchronous completion callback, so the Pi’s overlapped p50 tracks the service time (≈ 17 ms) at every rate, and the HTP pipeline submits rows as soon as the serialized call returns. On the HTP the shipped governor inflates latency instead, because the slowed feeder core stretches both the call and the overlapping host NMS (15 fps overlapped cells).

For a practitioner, the ordering of Section 10 survives with two amendments. On CPU-inference devices, stop after decode overlap with a single worker and spend the effort on clocks and cooling. On NPU devices, check the CPU governor early: on the QNN/HTP stack it was the largest single term, while the Hailo stack’s shipped governor was already within 1% of the pinned one at the best rung.

Answer to RQ5. The findings carry over in outline, not in mechanism. On all four stacks (Jetson TensorRT, Hailo-8, Rubik CPU, Rubik HTP), isolated engine or inference-call measurements understate the end-to-end application cost, for different reasons: a GPU-governor-induced engine slowdown on the Jetson (Section 5), host and runtime overhead around a fixed-function NPU on the Pi, shared CPU resources and thermal saturation on the Rubik CPU, and the CPU-side dispatch share of each call and its response to the CPU governor on the HTP. In each case the host-pipeline ladder recovers throughput without touching the engine, and streaming queues exactly when the arrival rate passes the ladder-measured capacity. Across model architectures (Section 8.3), YOLOv10n and YOLO26s (whose engine takes 1.6× longer per image) reproduce the same schedule effects (5.6–6.9× over the Ultralytics predictor), so the conclusions are not specific to YOLO26n.

10. Discussion

10.1. Implications for benchmarking

A back-to-back engine benchmark measures the GPU at a clock that a sequential application never reaches on this device, so engine-level speedups answer a different question

Table 16

Head configuration comparison on the Rubik HTP: the standard head with host NMS versus the NMS-free head fused into the INT8 context. This is not a clean ablation. The two contexts come from different QAIRT releases (2.49.40 versus 2.50.40) and different calibration sets, so the accuracy difference is a property of the two configurations, not an isolated cost of fusing selection. Accuracy is on all 5,000 val2017 images (same pipeline, serialized NPU calls); throughput is the accuracy run and the best ladder rung per governor. Under `schedutil` both best cells are the spinning-wait diagnostic (best non-spinning rungs: 27.8 and 36.9 images/s). Under performance the fused head’s spin and non-spin best rungs tie within run-to-run noise (63.2 versus 63.0).

Head	AP	AP ₅₀	AP _S	acc. img/s	best img/s (sched/perf)
standard (1,84,8400), host NMS	0.364	0.527	0.175	22.8	28.5 / 43.6
NMS-free, fused (3 outputs)	0.293	0.477	0.112	32.5	39.6 / 63.2

Table 17

Streaming on the NPU boards: p50 latency per frame in ms (release to finished rows; 1,000-frame streams on the Pi, 480-frame streams on the HTP). Pi: mean of two rotated repeats under the shipped `ondemand` governor, with SoC-side energy per frame in parentheses. HTP: one run per cell and governor (the two rotated repeats were assigned one per governor), so the governor contrast is paired but not replicated; the fused-head variant is discussed in the text. Queueing cells are rounded to two significant figures.

Board	Pipeline	Governor	15 fps	30 fps	60 fps
Pi 5 + Hailo-8	seq	ondemand	17.4 (102.0 mJ)	16.9 (61.0 mJ)	15.7 (37.4 mJ)
	full w2	ondemand	17.3 (111.1 mJ)	17.0 (58.7 mJ)	15.6 (36.2 mJ)
Rubik HTP	seq	schedutil	54.2	2400	6700
		performance	35.2	390	4700
	full w2	schedutil	108	820	5100
		performance	42.6	47	2100

from the one a deployment asks. The gap is not a constant factor; it depends on the schedule (Table 6), the arrival rate (Table 10) and the model’s own duty cycle (YOLO26n versus YOLO26s in Table 4). In these experiments it is not even monotone in the arrival rate. The uint8 and float sequential pipelines have higher median latency and higher CPU time per frame at 15 fps than at 30 fps, which points to a lower CPU clock during longer idle gaps (Section 6.3). This paper therefore suggests that efficiency claims for edge detectors report at least: (i) an end-to-end decode-to-detections measurement, (ii) the GPU and CPU governors and the power mode, (iii) the GPU and CPU clocks logged during the measurement, (iv) how the harness stores results, and (v) the workload scenario (backlog, fixed-rate stream or single frame), in the spirit of the MLPerf scenarios [28]. Locking clocks is a useful diagnostic, but not a neutral benchmarking choice: it removes exactly the interactions that dominate behavior under the stock governors a device ships with. Under locked clocks the synchronization call looks irrelevant on the sequential and prefetch schedules, while under the default governors it costs a fifth of the throughput. On the two-worker schedule only the synchronization boundary matters (24% for stream synchronization), identically in both regimes (Section 6.3). Locking also raises idle power by 1.9 W, which at camera rates costs more energy than any pipeline choice saves (Section 7).

10.2. Guidelines for practitioners

The ordering that worked on this platform was:

1. Take host stages off the GPU’s critical path first (decode prefetch, overlapped post-processing). Choose a wait that does not let the CPU governor lower the host’s clock: a spinning wait or, where a sleeping wait induces frequency collapse, a per-task utilization clamp, which needed no root privileges on the test system and costs less energy (Section 6.3). Once the pipeline keeps the host busy, the wait policy no longer matters, but the synchronization boundary does. In an enqueue-ahead loop, synchronize on per-frame events, not on the whole stream (Table 9).
2. Move preprocessing into the engine once the CPU becomes the bottleneck. Its value grows from 18% to 2.4× as the schedule improves.
3. Match the schedule to the workload. Use overlap with several decode workers for backlogs and high rates. Use the arrival-aware variant when frames arrive at camera rates and latency matters. Do not lock clocks for a camera application that runs below capacity.
4. Only then evaluate model-level savings (resolution, pruning, quantization), in the optimized pipeline and with application-level metrics.

Step 4 matters because the pipeline effect is larger than the model effect. In these measurements, the full YOLO26s pipeline under default governors is faster and more accurate than the Ultralytics YOLO26n predictor with locked clocks (Table 11). Model sizes should be compared within the same, optimized pipeline.

10.3. Why compression is not useless

The negative compression results are specific to a 2.6 M-parameter detector whose compiled engine takes about 4 ms, on a device whose host stages cost more than the engine at full clock. For larger models whose engine is the bottleneck even in an optimized pipeline (YOLO26s reaches 95% of its back-to-back engine rate), reducing engine work translates directly into throughput, and compression regains its usual role. The results argue for evaluating compression in the regime where it will be deployed, not for abandoning it.

10.4. Relation to frequency-control research

DVFS research on edge inference usually controls the frequency directly [15–17]. The present results are complementary: with the stock governor left in place, the application’s own schedule determines the operating point, and a well-scheduled pipeline lets the governor choose the top clock on its own. A governor-aware design could go further, for example by deliberately running at the lowest clock that meets a latency target at a known frame rate. The 60 fps streaming results show the potential: the governor ran the optimized pipelines at 405–430 MHz, and their marginal energy per frame fell below the backlog value. The CPU results add a second dimension: the CPU and GPU governors act independently, but through the host pipeline the CPU clock sets the GPU’s duty cycle and therefore its clock (Section 6.3). A coordinated policy would have to treat the two as one control problem. Utilization clamping is a first, coarse step in this direction: it lets the application tell the CPU governor that its sleeping thread is on the GPU’s critical path.

11. Threats to validity

Internal validity. Scheduling variants give byte-identical predictions, so accuracy is not a confound. Run order was rotated, the page cache was warmed before each phase, and clock state was stored, restored and verified. Thermal state was controlled with 10 s pauses and junction-temperature logging. The Jetson peaked at 67.4 °C in application runs and 69.2 °C under `trtexec`, far from throttling. The Rubik CPU runs, by contrast, sit at the 95 °C thermal limit and show a 17% drift (Section 9.2). Repeats per cell are few, and 20 Hz power and clock sampling cannot resolve sub-50 ms clock changes (Section 5). A direct `devfreq` lock of the memory (EMC) clock did not hold on this release, a silent failure also reported on this board family [21]. In the fixed-frequency sweep it was therefore logged and its influence bounded to 2% of the fitted slope. `jetson_clocks` does hold it (Section 4), so locked-versus-default comparisons include any memory-clock effect. Runs last 9–203 s on the Jetson and ≈ 26 min for the Rubik CPU sweep, so deployment-length steady state was not tested. The ramp in Fig. 4 shows that a run’s first seconds differ from the rest. The Ultralytics baseline uses its own engine export. Its AP agrees to 3×10^{-5} , but engine tactics may differ slightly.

Construct validity. `VDD_IN` is the module’s main input rail. It excludes carrier-board loads (fan, USB peripherals, storage regulators) but overstates the energy of the SoC compute rails alone. The idle cost of locking clocks includes the higher memory, CPU and GPU clocks. The on-board sensor was not validated against an external meter, so comparisons rely on its run-to-run consistency, not its absolute accuracy. Latency starts when a JPEG file is available, so it includes decode but not camera capture, ISP processing or output transmission. Process CPU time includes all threads.

External validity. The main study uses one Jetson Orin Nano Super and one software stack, only partially offset by the replications of Section 9. Other Jetson modules, power modes, JetPack releases, hardware JPEG decode [25] or DeepStream pipelines may shift the magnitudes, and other vendors’ governors may differ. Pi 5 energies cover the SoC only, because the Hailo-8 sits on the unmonitored 5 V rail. Its on-chip NMS uses a 0.2 score threshold, so Pi accuracy is not comparable with the other boards. The Rubik Pi 3 has no usable power sensor. Its NPU stack uses a third-party runtime (`qai-appbuilder`) rather than the vendor SDK. Its INT8 model ends in the standard head with host-side NMS, so accuracy is not comparable with the float stacks (Section 9.3). That userspace wedged under concurrent calls, so the NPU pipeline serializes accelerator calls and overlaps only host stages. Multi-threaded CPU inference emits rows in nondeterministic order, so prediction equivalence there is shown on sorted rows (identical in all 24 ladder cells), not by a byte-level hash. The pipelines are written in Python; native C++ or DeepStream would shorten the host stages and could reach the GPU-bound regime with fewer decode workers. Batch size was 1 throughout, as in a camera application; its known interaction with GPU frequency [14] was not measured. The compression screening (one seed, short recovery, `opt0` builds) is evidence about that budget, not a general verdict on pruning or quantization.

12. Conclusion

On an embedded GPU, the deployed cost of an NMS-free detector is not a property of the model alone. Under the stock governor, the same YOLO26n engine costs 4.2 ms or 13.2 ms depending on how busy the application keeps the GPU. A simple duty-cycle model describes this behavior. Restructuring only the host pipeline, under default governors and at unchanged accuracy, gave 2.4× the throughput of the best sequential pipeline at locked maximum clocks.

The CPU governor is part of the same loop. On the sequential and prefetch schedules, a sleeping CUDA wait lets it lower the host clock and, through slower submission, the GPU clock. A per-task utilization clamp recovers most of this loss without busy-spinning. On the full two-worker pipeline the wait policy costs nothing under default and locked clocks. Only the stream-synchronization boundary still costs 24%.

Below capacity, idle power sets energy per frame and the schedule sets latency. Near capacity the schedule decides

energy again, so the right schedule depends on the arrival rate. Model-level decisions matter much less in an optimized pipeline. A lower input resolution gave 27% more throughput in the naive pipeline but only 2–4% in the optimized one (6–10% with the garbage collector disabled). Compression and routing gained less than scheduling.

The effect transfers to a Raspberry Pi 5 with a Hailo-8 NPU and to a Qualcomm board, but its mechanism moves to the host. The Hailo-8 exposes no frequency control, the Qualcomm CPU stack is bound by thermal management, and on its Hexagon NPU the scheduler down-clocks the core that feeds the accelerator. Edge-detection efficiency should thus be reported for the whole pipeline under deployment governors.

A. Compression screening and router details

Before the pipeline study, most of the effort went into the model. Table A.1 summarizes the screening, all of which used a 1,024-image development split of COCO train2017, disjoint from val2017, with official COCOeval. Each method received the same short recovery budget: 1,024 training images, five epochs and one seed.

Channel pruning removed 25% of the output channels of four backbone convolutions, or 5.6% of the parameters. The proposed object-risk criterion scores channels by gradient-weighted foreground activations and a fake-INT8 perturbation proxy. At the same budget it kept 2.7 AP points more than L1 magnitude pruning, but both pruned models still lost 6.8–9.5 AP points. Neither was faster after compilation, because TensorRT does not spend its time in the pruned early convolutions. Ultralytics QAT produced a mixed-precision engine (about half of the convolutions in INT8, the rest FP16/FP32 behind reformatting layers). It was slower than FP16 under both builder levels. Distillation from the fine-tuned teacher reduced AP in both pruning settings (L1: 0.369 to 0.338; object-risk: 0.395 to 0.373 on the development split). Pruning the parts that per-layer profiles flagged as expensive (P3/P4 classification-head channels and the attention block’s feed-forward width) gave no stable change in engine time beyond run-to-run noise. That engine time is about 6.5 ms back-to-back with opt0 builds. The table’s 21.8 ms column is the same engine inside a sequential application, and the difference between the two is the governor effect of Section 5. These results come from the short budget above, with opt0 builds on one device. Longer training would likely let the pruned models recover more AP, and a full opt3 build sweep might redistribute the engine time. The results therefore do not show that compression cannot work. They show that, for a 2.6 M-parameter NMS-free detector on this device and budget, the achievable engine-time savings are a small fraction of the 3× governor effect of Section 5.

A.1. The per-image resolution router

A logistic router was fitted on cheap pre-inference image statistics. It chooses a 512 or 640 engine per image, with its threshold fixed to send 20% of images to 640. On its 512-image held-out half, it slightly beat a fixed 576 engine in both AP and median time. It was then frozen and evaluated once on an independent, hash-verified 1,024-image train2017 split that played no part in fitting or threshold selection (Section 4). Against fixed 576 it lost 0.0089 AP (0.4386 vs. 0.4476) and 0.0147 AP_S, and both 512-image halves lost AP. Its median application time was 1.1–1.4 ms lower across three paired runs (41.3–41.9 versus 42.4–43.2 ms), but its 95th-percentile time was not. Small-object recall at IoU 0.5 fell by 1.55 points (paired image bootstrap 95% interval [−2.98, −0.19]). The router was therefore rejected. In the same units, the scheduling steps of Table 6 save 15–28 ms of median per-image latency, depending on the starting pipeline. The router’s ≈1.2 ms is at most one eleventh of that. Latency rather than period is compared because the router was evaluated in a sequential

Table A.1

Compression screening for YOLO26n (development split of 1,024 train2017 images, one seed, five recovery epochs, TensorRT 10.3). The two blocks come from different measurement contexts and are not directly comparable. Block (a): median host-measured copy+execute+copy per image in a sequential application with opt0 builds, which includes the down-clocking of Section 5. Block (b): back-to-back trtexec GPU compute with opt3 builds (driver keeps the GPU near 1,020 MHz; AP_S not recorded in that harness).

Variant	AP	AP _S	ms
<i>(a) sequential application, opt0 build</i>			
FP16, fine-tuned (reference)	0.4691	0.2164	21.8
Mixed QAT (50/99 conv INT8)	0.4532	0.2038	24.4
L1 channel pruning (−5.6% params)	0.3741	0.1426	21.8
Object-risk pruning (−5.6% params)	0.4007	0.1613	21.8
L1 pruning + QAT	0.3612	0.1372	24.3
Object-risk pruning + QAT	0.3924	0.1594	24.5
<i>(b) back-to-back trtexec, opt3 build</i>			
FP16	0.4686	–	4.07
Mixed QAT	0.4593	–	5.00

pipeline, so its saving does not compose with an overlapped schedule.

B. QNN userspace notes

Two findings about the bundled qai-appbuilder/QNN userspace (QAIRT 2.50.40 wheel on QCM6490, V68 HTP) shaped the HTP campaign. First, it is not safe under concurrent inference. Calls from two contexts at once wedge the DSP session after a few hundred calls, leaving an empty output or a session that never returns; retries and context rebuilds do not recover it. This applies only to the bundled userspace as shipped. The vendor SDK advertises multi-context support, which was not exercised. The pipeline therefore holds exactly one context and issues every NPU call from a single thread. Overlap comes from pipelining decode, input fill and host NMS *around* the serialized NPU calls. Under that discipline the campaign ran without a failed call.

Second, an earlier version of the evaluation script contained a harness bug. It passed the input buffer to the appbuilder wrapper unflattened, and the wrapper’s reshape helper caught the resulting broadcast error and printed it on every call, inside the timed region. This cost ≈1 ms per call at the shipped governor and more at pinned low clocks. Predictions were unaffected, because the native call received the correct buffer. After the call was fixed, the entire HTP campaign was rerun, and only the corrected numbers are reported. The fix moved the shipped-governor rungs by ≈1 ms per call, the performance ladder by up to 13% (the pinned core now spends its cycles on dispatch instead of printing), and the 806 MHz probe point by 8 ms per call. It therefore mattered most where the paper examines clock sensitivity.

Data and code availability

The measurement scripts (pipeline, Ultralytics baseline, power sampler, campaign runner, aggregation and figure generation), the raw timestamped traces (Jetson: power, clocks, GPU load and temperature at 20 Hz; Pi 5: PMIC power at ≈ 10 Hz and clocks; Rubik: clocks at 13–18 Hz, no power rail), per-run reports, SHA-256 hashes of all prediction files, and the COCOeval summaries, and the aggregation and plotting scripts that generate the tables and figures from these traces are provided at <https://github.com/iamdinhthuan/governor-is-part-of-the-model>, whose README maps every table and figure to its data and generator script, so nearly every reported number can be recomputed; a small number of numbers remain verbatim in the LaTeX source (the Table A.1 screening table and the prose router results, whose aggregators are included). The prediction files themselves (1.6 GB uncompressed), whose SHA-256 hashes anchor the byte-identical-prediction claims, are archived on Zenodo [41] together with the campaign traces; traces of the control runs added after that deposit are in the GitHub repository.

CRedit authorship contribution statement

Dinh Thuan Nguyen: Conceptualization, Methodology, Software, Investigation, Formal analysis, Writing—original draft. **Lam Phuong Nguyen:** Validation, Data curation, Writing—review and editing. **Vinh Huy Nguyen:** Methodology, Visualization, Writing—review and editing. **Sy Vu Quang:** Validation, Writing—review and editing. **Mohan Rajesh Elara:** Supervision, Writing—review and editing. **Anh Vu Le:** Supervision, Project administration, Writing—review and editing.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Factory Droid to run the measurement campaigns and assist with drafting. The authors reviewed and edited all content and take full responsibility for it.

References

- [1] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, C. L. Zitnick, Microsoft COCO: Common objects in context, in: European Conference on Computer Vision (ECCV), 2014, pp. 740–755.
- [2] H. Li, A. Kadav, I. Durdanovic, H. Samet, H. P. Graf, Pruning filters for efficient ConvNets, in: International Conference on Learning Representations (ICLR), 2017.
- [3] G. Fang, X. Ma, M. Song, M. B. Mi, X. Wang, DepGraph: Towards any structural pruning, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023.
- [4] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, D. Kalenichenko, Quantization and training of neural networks for efficient integer-arithmetic-only inference, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2018, pp. 2704–2713.
- [5] G. Hinton, O. Vinyals, J. Dean, Distilling the knowledge in a neural network, arXiv preprint arXiv:1503.02531 (2015).
- [6] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, G. Ding, YOLOv10: Real-time end-to-end object detection, in: Advances in Neural Information Processing Systems (NeurIPS), 2024.
- [7] G. Jocher, J. Qiu, M. Liu, S. Lyu, F. C. Akyon, M. E. Kalfaoglu, Ultralytics YOLO26: Unified real-time end-to-end vision models, arXiv:2606.03748 [cs.CV] (2026).
- [8] Ultralytics, Ultralytics YOLO26 documentation, <https://docs.ultralytics.com/models/yolo26/>, software release 8.4.172 used in this work (2026).
- [9] The Linux Kernel Documentation, Utilization clamping, <https://docs.kernel.org/scheduler/sched-util-clamp.html>, accessed 2026-10-05 (2024).
- [10] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, S. Zagoruyko, End-to-end object detection with transformers, in: European Conference on Computer Vision (ECCV), 2020, pp. 213–229.
- [11] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, J. Chen, DETRs beat YOLOs on real-time object detection, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024.
- [12] L. L. Zhang, S. Han, J. Wei, N. Zheng, T. Cao, Y. Yang, Y. Liu, nn-Meter: Towards accurate latency prediction of deep-learning model inference on diverse edge devices, in: ACM International Conference on Mobile Systems, Applications, and Services (MobiSys), 2021.
- [13] Z. Tang, Y. Wang, Q. Wang, X. Chu, The impact of GPU DVFS on the energy and performance of deep learning: An empirical study, in: Proceedings of the Tenth ACM International Conference on Future Energy Systems (e-Energy), 2019.
- [14] S. M. Nabavinejad, S. Reda, M. Ebrahimi, Coordinated batching and DVFS for DNN inference on GPU accelerators, IEEE Transactions on Parallel and Distributed Systems 33 (10) (2022).
- [15] S. Kim, K. Bin, S. Ha, K. Lee, S. Chong, zTT: Learning-based DVFS with zero thermal throttling for mobile devices, in: ACM International Conference on Mobile Systems, Applications, and Services (MobiSys), 2021.
- [16] Y. Han, Z. Nan, S. Zhou, Z. Niu, DVFS-aware DNN inference on GPUs: Latency modeling and performance analysis, arXiv preprint arXiv:2502.06295 (2025).
- [17] Z. Zhang, Z. Wu, J. Liu, L. Mottola, SparseDVFS: Sparse-aware DVFS for energy-efficient edge inference, arXiv preprint arXiv:2603.21908 (2026).
- [18] A. Pathania, Q. Jiao, A. Prakash, T. Mitra, Integrated CPU-GPU power management for 3D mobile games, in: Proceedings of the 51st Annual Design Automation Conference (DAC), 2014. doi: 10.1145/2593069.2593151.
- [19] M. Karzhaubayeva, A. Amangeldi, J.-G. Park, CNN workloads characterization and integrated CPU–GPU DVFS governors on embedded systems, IEEE Embedded Systems Letters 15 (4) (2023) 202–205. doi:10.1109/LES.2023.3299335.

- [20] Y. Gong, Y. Wu, Z. Zhan, P. Zhao, L. Liu, C. Wu, X. Tang, Y. Wang, Lotus: Learning-based online thermal and latency variation management for two-stage detectors on edge devices, in: 61st ACM/IEEE Design Automation Conference (DAC), 2024. doi:10.1145/3649329.3657310.
- [21] J. Kang, Edge-inference governors need memory-clock state, arXiv:2606.16106v3 [cs.PF] (2026).
- [22] S. Mittal, A survey on optimized implementation of deep learning models on the NVIDIA Jetson platform, *Journal of Systems Architecture* 97 (2019) 428–442. doi:10.1016/j.sysarc.2019.01.011.
- [23] R. Hadidi, J. Cao, Y. Xie, B. Asgari, T. Krishna, H. Kim, Characterizing the deployment of deep neural networks on commercial edge devices, in: Proceedings of the IEEE International Symposium on Workload Characterization (IISWC), 2019.
- [24] D. Richins, D. Doshi, M. Blackmore, A. T. Nair, N. Pathapati, A. Patel, B. Daguman, D. Dobrijalowski, R. Illikkal, K. Long, D. Zimmerman, V. J. Reddi, AI tax: The hidden cost of AI data center applications, *ACM Transactions on Computer Systems* 37 (1–4) (2021) 3:1–3:32. doi:10.1145/3440689.
- [25] T. Chen, N. Alnaasan, J. Yao, A. Shafi, H. Subramoni, D. K. Panda, Accelerating data preprocessing for efficient vision model inference on Jetson edge device, arXiv preprint arXiv:2608.27655 (2026).
- [26] P. S. K., S. A. Kesanapalli, Y. Simmhan, Characterizing the performance of accelerated Jetson edge devices for training deep learning models, *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 7 (1) (2023). doi:10.1145/3570604.
- [27] Q. Zhang, F. Machida, L. Carnevali, Input resolution matters: Real-time object detection latency, in: IEEE International Symposium on Modelling, Analysis and Simulation of Computer and Telecommunication Systems (MASCOTS), 2026, arXiv:2609.12920.
- [28] V. J. Reddi, C. Cheng, D. Kanter, P. Mattson, G. Schmuelling, C.-J. Wu, B. Anderson, M. Breughe, M. Charlebois, W. Chou, R. Chukka, C. Coleman, S. Davis, P. Deng, G. Diamos, J. Duke, D. Fick, J. S. Gardner, I. Hubara, S. Idgunji, T. B. Jablin, J. Jiao, T. S. John, P. Kanwar, D. Lee, J. Liao, A. Lokhmotov, F. Massa, P. Meng, P. Micikevicius, C. Osborne, G. Pekhimenko, A. T. R. Rajan, D. Sequeira, A. Sirasao, F. Sun, H. Tang, M. Thomson, F. Wei, E. Wu, L. Xu, K. Yamada, B. Yu, G. Zhang, A. Zhong, P. Zhou, Y. Zhu, MLPerf inference benchmark, in: ACM/IEEE International Symposium on Computer Architecture (ISCA), 2020, pp. 446–459.
- [29] W. Jang, H. Jeong, K. Kang, N. Dutt, J.-C. Kim, R-TOD: Real-time object detector with minimized end-to-end delay for autonomous driving, in: IEEE Real-Time Systems Symposium (RTSS), 2020, arXiv:2011.06372.
- [30] T.-W. Chin, R. Ding, D. Marculescu, AdaScale: Towards real-time video object detection using adaptive scaling, in: Conference on Machine Learning and Systems (SysML), 2019.
- [31] M. Zhu, K. Han, E. Wu, Q. Zhang, Y. Nie, Z. Lan, Y. Wang, Dynamic resolution network, in: Advances in Neural Information Processing Systems (NeurIPS), 2021.
- [32] NVIDIA, Jetson Linux R36.5.2 public sources, kernel linux-tegra-5.15, driver drivers/video/tegra/host/nvhost_podgov.c, https://developer.nvidia.com/downloads/embedded/14t/r36_release_v5.2/sources/public_sources.tbz2 (2025).
- [33] NVIDIA, Jetson Linux developer guide R36.5.2: Platform power and performance (Jetson Orin Nano, Orin NX and AGX Orin series), <https://docs.nvidia.com/jetson/archives/r36.5.2/DeveloperGuide/SD/PlatformPowerAndPerformance/JetsonOrinNanoSeriesJetsonOrinNXSeriesAndJetsonAgxOrinSeries.html> (2026).
- [34] Hailo, Hailo model zoo: pre-trained models and pre-compiled HEF files for Hailo-8 accelerators, https://github.com/hailo-ai/hailo_model_zoo (2023).
- [35] Microsoft, ONNX Runtime: cross-platform inference and training accelerator, <https://github.com/microsoft/onnxruntime>, version 1.30.0 used in this work, accessed 2026-10-05 (2026).
- [36] NVIDIA, TensorRT 10.3 developer guide, <https://docs.nvidia.com/deeplearning/tensorrt/> (2024).
- [37] Python Software Foundation, Design of CPython’s garbage collector (Python developer’s guide), <https://devguide.python.org/internals/garbage-collector/> (2024).
- [38] NVIDIA, CUDA runtime API: Device management (cudasetdevice-flags), <https://docs.nvidia.com/cuda/cuda-runtime-api/> (2024).
- [39] R. J. Wysocki, CPU performance scaling (Linux kernel documentation, schedutil governor), <https://docs.kernel.org/admin-guide/pm/cpufreq.html>, living document, accessed 2026-10-05 (2026).
- [40] Ultralytics, Ultralytics YOLOv8 documentation, <https://docs.ultralytics.com/models/yolov8/>, accessed 2026-10-05 (2023).
- [41] D. T. Nguyen, L. P. Nguyen, V. H. Nguyen, S. V. Quang, M. R. Elara, A. V. Le, Raw measurement data: The governor is part of the model (YOLO26n edge-inference campaigns), [dataset] (2026). doi:10.5281/zenodo.23185860.